

UNIVERSIDADE FEDERAL DO RIO GRANDE DO SUL
ESCOLA DE ENGENHARIA
ENGENHARIA DE CONTROLE E AUTOMAÇÃO

MAXWELL RODRIGUES DA SILVA - 282162

**APLICAÇÃO E AVALIAÇÃO DE MACHINE
LEARNING EM DISPOSITIVOS
EMBARCADOS**

Porto Alegre
2025

UNIVERSIDADE FEDERAL DO RIO GRANDE DO SUL
ESCOLA DE ENGENHARIA
ENGENHARIA DE CONTROLE E AUTOMAÇÃO

MAXWELL RODRIGUES DA SILVA - 282162

**APLICAÇÃO E AVALIAÇÃO DE MACHINE
LEARNING EM DISPOSITIVOS
EMBARCADOS**

Trabalho de Conclusão de Curso submetido à
COMGRAD/CCA da UFRGS como parte dos requi-
sitos para a obtenção do título de *Bacharel em Enge-
nharia de Controle e Automação*.

Orientador:
Prof. Dr. Marcelo Götz

Porto Alegre
2025

UNIVERSIDADE FEDERAL DO RIO GRANDE DO SUL
ESCOLA DE ENGENHARIA
ENGENHARIA DE CONTROLE E AUTOMAÇÃO

MAXWELL RODRIGUES DA SILVA - 282162

**APLICAÇÃO E AVALIAÇÃO DE MACHINE
LEARNING EM DISPOSITIVOS
EMBARCADOS**

Este Trabalho de Conclusão de Curso foi julgado adequado para a obtenção dos créditos da Atividade de *Trabalho de Conclusão de Curso CCA - II* e aprovado em sua forma final pelo Orientador e Banca Examinadora abaixo.

Orientador:

Prof. Dr. Marcelo Götz, UFRGS

Doutor pela Universität Paderborn - Paderborn, Alemanha

Banca Examinadora:

Prof. Dr. Marcelo Götz, UFRGS

Doutor pela Universität Paderborn - Paderborn, Alemanha

Prof. Dr. Rafael Antônio Comparsi Laranja, UFRGS

Doutor pela Universidade Federal do Rio Grande do Sul – Porto Alegre, Brasil

Prof. Dr. Valner João Brusamarello, UFRGS

Doutor pela Universidade Federal de Santa Catarina – Florianópolis, Brasil

Fabiano Disconzi Wildner

Coordenador de Curso

Eng. de Controle e Automação

Porto Alegre

Junho - 2025

DEDICATÓRIA

Dedico este trabalho à minha mãe e ao meu pai por todo apoio, ao Bruno, o melhor irmão que alguém poderia ter, a todos meus queridos amigos que alegram minha vida e em especial à Jaque, minha esposa, pela dedicação e apoio em todos os momentos difíceis, por sempre estar ao meu lado e pelo nosso amor.

AGRADECIMENTOS

À Universidade Federal do Rio Grande do Sul (UFRGS), pela oportunidade de estudar em uma das instituições mais renomadas da América Latina e do mundo.

A todos que lutam e lutaram pela implementação da Lei de Cotas no Brasil, um marco na construção de um país mais justo e que possibilitou minha formação acadêmica.

Ao Laboratório de Metalurgia Física (LAMEF), por ter me dado a oportunidade de iniciar minha trajetória profissional, onde tive a chance de aprender e crescer.

À minha liderança Eng. Fábio Kipper, e também ao Eng. Cristiano Stefaniaki e Daniel Dias, por me receberem na TK Elevadores e pela compreensão e flexibilidade diante dos horários desafiadores do curso de Engenharia da UFRGS, conciliados com as demandas do trabalho. Agradeço também ao Lucas Schmidt, meu colega na TK Elevadores, pela ajuda prestada ao emprestar itens e ferramentas para este trabalho.

E, por fim, ao corpo docente da Escola de Engenharia desta Universidade, que, mesmo recorrendo, por vezes, a métodos desafiadores e pouco ortodoxos, contribuiu significativamente para minha formação como profissional.

RESUMO

Com a escalada da demanda por dispositivos da Internet das Coisas (IoT) e a necessidade crescente de inteligência embarcada, este trabalho avaliou o estado da arte do TinyML e investigou sua aplicabilidade prática em sistemas com recursos limitados. Foi realizado um levantamento e análise de ferramentas e plataformas TinyML, seguido da proposição de um fluxo de trabalho ponta-a-ponta para desenvolvimento (aquisição e pré-processamento de dados, treinamento, quantização, validação e implantação). Como estudo de caso, implementou-se um classificador de resíduos sólidos: adotou-se o conjunto RealWaste, aplicou-se balanceamento via data augmentation, treinou-se uma MobileNetV2 em TensorFlow com ajuste fino, e converteu-se o modelo para TensorFlow Lite com quantização INT8. Para validação prática, o modelo quantizado foi implantado e testado em uma Raspberry Pi 3 Model B+, medindo latência, uso de CPU e consumo de memória. Os resultados mostram que a solução é funcional em ambiente embarcado (acurácia global 0,68 no conjunto de validação; latência média 200 ms por inferência; uso de memória do processo 85 MB), evidenciando compromissos entre precisão e restrições de hardware. Conclui-se que TinyML é viável para reduzir latência e dependência de nuvem em aplicações reais, embora melhorias em dados (mais imagens e padronização de captura), ajustes de augmentations e exploração de arquiteturas/quantizações mais adequadas possam elevar a acurácia e robustez do sistema.

Palavras-chave: TinyML, aprendizado de máquina, computação de borda, internet das coisas, sistemas embarcados

ABSTRACT

With the escalating demand for Internet of Things (IoT) devices and the growing need for embedded intelligence, this work evaluated the state of the art of TinyML and investigated its practical applicability in resource-constrained systems. A survey and analysis of TinyML tools and platforms was conducted, followed by the proposal of an end-to-end workflow for development (data acquisition and preprocessing, training, quantization, validation, and deployment). As a case study, a solid waste classifier was implemented: the RealWaste dataset was adopted, balancing via data augmentation was applied, a MobileNetV2 was trained in TensorFlow with fine-tuning, and the model was converted to TensorFlow Lite with INT8 quantization. For practical validation, the quantized model was deployed and tested on a Raspberry Pi 3 Model B+, measuring latency, CPU usage, and memory consumption. The results show that the solution is functional in an embedded environment (overall accuracy 0.68 in the validation set; average latency 200 ms per inference; process memory usage 85 MB), highlighting trade-offs between accuracy and hardware constraints. We conclude that TinyML is viable for reducing latency and cloud dependency in real applications, although improvements in data (more images and capture standardization), adjustments to augmentations, and exploration of more suitable architectures/quantizations could increase the accuracy and robustness of the system.

Palavras-chave: TinyML, edge computing, embedded systems, internet of things, machine learning

LISTA DE ILUSTRAÇÕES

1	Exemplo de uma rede neural simples.	17
2	Exemplo de um gráfico COR.	21
3	Exemplo de uma Matriz de Confusão.	22
4	Processo de seleção manual de resíduos.	25
5	Exemplos de imagens do conjunto de dados <i>RealWaste</i> : (a) papel; (b) metal; (c) vidro; (d) plástico; (e) resíduo orgânico alimentar; (f) resíduo têxtil.	29
6	Curvas de acurácia e perda por época (treino e validação).	32
7	Matriz de confusão normalizada (%).	38
8	Curvas COR por classe e respectivas AUCs.	39
9	Montagem utilizada nos ensaios, composta por tubos de PVC, presilhas de fixação, webcam USB e Raspberry Pi.	40
10	Ensaio com tampa metálico de Ø 160 mm mostrando a predição da classe e o tempo de inferência.	42
11	Ensaio com papel A4, ilustrando o tempo de inferência registrado e a classificação produzida pelo modelo.	42
12	Ensaio com caixas de medicamento (papelão), exibindo o tempo de inferência e a predição realizada.	43
13	Ensaio com toalha de banho (têxtil), evidenciando tempo de inferência e classificação final.	43
14	Ensaio de controle com modelo Float32: latência equivalente (183 ms), porém com aumento significativo no consumo de memória (+27%) devido à precisão de 32 bits.	45

LISTA DE TABELAS

1	Modelos de aprendizado de máquina e suas principais funções no contexto de TinyML.	17
2	Principais funções de perda para diferentes categorias de aprendizado de máquina.....	18
3	Especificações de dispositivos TinyML de baixo e alto desempenho....	23
4	Comparação de desempenho entre TensorFlow e Edge Impulse.	23
5	Comparação de métricas de desempenho entre TensorFlow e Edge Impulse.....	24
6	Categorias e quantidade de imagens no conjunto de dados RealWaste..	28
7	Especificações principais da Raspberry Pi 3 Model B+.	34
8	Métricas globais (validação) — modelo TFLite INT8.....	36
9	Relatório de classificação por classe (validação).....	38
10	Lista de materiais para a estrutura de suporte (PVC 1/2").....	41

SUMÁRIO

1	INTRODUÇÃO	12
1.1	Objetivos gerais	12
1.2	Objetivos específicos	13
1.3	Hipótese	13
1.4	Estrutura do trabalho	13
2	REVISÃO BIBLIOGRÁFICA	14
2.1	TinyML e Dispositivos Embarcados	14
2.2	Trabalhos Relacionados e Abordagens na Literatura	14
2.3	Considerações Finais da Revisão	15
3	FUNDAMENTAÇÃO TEÓRICA	16
3.1	Aquisição e Preparação dos Dados	16
3.2	Desenvolvimento e Treinamento de Modelos	17
3.3	Quantização para Compactação de Modelos	18
3.3.1	Formulação Matemática	19
3.3.2	Tipos de Quantização	19
3.3.3	Impacto na Acurácia e Desempenho	20
3.4	Validação, Ajuste Fino e Métricas de Desempenho	20
3.5	Implantação em Dispositivos Embarcados	22
4	ESTUDO DE CASO: CLASSIFICAÇÃO DE RESÍDUOS SÓLIDOS...	25
4.1	Descrição do estudo de caso	25
4.2	Etapas do estudo de caso	26
5	DESENVOLVIMENTO	28
5.1	Coleta e seleção do banco de dados	28
5.2	Pré-processamento dos dados	29
5.3	Desenvolvimento e treinamento do modelo	30
5.3.1	Fluxo de dados	30
5.3.2	Arquitetura e estratégia de treinamento	30
5.3.3	Função de perda, otimizador e mecanismos de controle (<i>callbacks</i>)	31
5.3.4	Resultados do treino	31
5.4	Quantização do modelo	32
5.4.1	Abordagem adotada	32
5.4.2	Validação do modelo quantizado	33

5.5	Escolha e preparação do hardware	33
5.5.1	Preparação e configuração inicial	34
5.6	Implementação e demonstração prática	34
5.6.1	Fluxo de inferência	35
6	RESULTADOS E DISCUSSÃO	36
6.1	Avaliação de desempenho	36
6.1.1	Métricas globais	36
6.1.2	Comparação com o estudo de referência	37
6.1.3	Relatório de classificação por classe	37
6.1.4	Matriz de confusão	38
6.1.5	Curvas COR multiclasse	39
6.1.6	Discussão	39
6.2	Avaliação e análise de eficiência computacional	39
6.2.1	Montagem experimental	40
6.2.2	Resultados sobre o tempo de inferência	41
6.2.3	Uso de CPU e memória RAM	43
6.2.4	Análise comparativa: Quantizado (INT8) vs. Ponto Flutuante (Float32)	44
6.2.5	Síntese dos resultados	45
6.3	Limitações e sugestões de melhoria	46
7	CONCLUSÕES	47
	REFERÊNCIAS	49

1 INTRODUÇÃO

O avanço da tecnologia e a popularização dos dispositivos conectados têm impulsionado o desenvolvimento de soluções inteligentes em diversas áreas. Nesse cenário, a Internet das Coisas (IoT) destaca-se como um dos principais motores da transformação digital, permitindo a integração de sensores, atuadores e sistemas embarcados em aplicações que vão da saúde à indústria. No entanto, desafios como a latência na comunicação com a nuvem e a limitação de recursos computacionais em dispositivos embarcados exigem novas abordagens para garantir eficiência e desempenho em tempo real (XAVIER *et al.*, 2022).

O número de dispositivos pertencentes à IoT tem apresentado um crescimento constante durante a terceira década do século XXI (SINHA, 2024). Esses dispositivos estão sendo implementados em diversas áreas, como saúde (SHWETHA; SWETHA, 2024), agronomia (KJELLBY *et al.*, 2019) e indústria (DEMIR; KORKMAZ, 2023). Esse cenário reforça a importância de estudar soluções eficientes para IoT, já que seu impacto se estende a áreas essenciais do cotidiano e do desenvolvimento econômico.

Uma limitação dos dispositivos IoT é a latência do envio de dados para centros de processamento remotos. Mesmo os melhores serviços de nuvem alcançam latências de até 28 ms (ZEN *et al.*, 2022), variando conforme horário e tráfego de dados. Essa latência é um desafio em aplicações que exigem decisões em tempo real, como controle crítico, automação industrial e dispositivos de saúde (OLADOKUN, 2025).

A tecnologia de aprendizado de máquina aplicada a sistemas embarcados, conhecida como TinyML, permite realizar o processamento local dos dados no próprio dispositivo. Esse paradigma, associado à computação de borda, reduz a dependência de processamento remoto e melhora significativamente a latência, com reduções de 35% a 40% conforme indicado em (SARASWATHI, 2025), tornando-a mais adequada para aplicações em tempo real.

Avaliar o estado atual de desenvolvimento do TinyML e um fluxo de trabalho para o desenvolvimento de soluções utilizando dispositivos embarcados pode contribuir para superar os desafios relacionados à dependência de conectividade. Além disso, tal abordagem viabiliza o desenvolvimento de sistemas mais robustos, eficientes e adaptados a aplicações críticas ou em áreas remotas. Com base nisso, este trabalho propõe-se a investigar, de forma estruturada, até que ponto as soluções baseadas em TinyML podem, atualmente, substituir ou complementar arquiteturas IoT tradicionais — conforme detalhado nos objetivos específicos a seguir.

1.1 OBJETIVOS GERAIS

Este trabalho tem como objetivo avaliar a viabilidade de aplicar soluções baseadas em aprendizado de máquina a sistemas embarcados, com foco em dispositivos caracterizados por recursos computacionais limitados.

1.2 OBJETIVOS ESPECÍFICOS

Os objetivos específicos deste trabalho são os seguintes:

1. Pesquisar e avaliar as ferramentas de desenvolvimento TinyML disponíveis, considerando critérios como facilidade de uso e integração com as plataformas mais populares;
2. Realizar o levantamento e a análise das plataformas disponíveis para projetos TinyML, considerando aspectos como desempenho, consumo energético e custo;
3. Elaborar um fluxo de trabalho ponta-a-ponta para desenvolvimento de soluções baseadas em técnicas TinyML, de forma a ilustrar o processo;
4. Desenvolver e aplicar, como estudo de caso, um modelo TinyML para classificação de resíduos sólidos, comparando seu desempenho com alternativas tradicionais.

1.3 HIPÓTESE

A aplicação de técnicas de TinyML em dispositivos embarcados com recursos computacionais limitados é viável e prática nos dias atuais, devido à disponibilidade crescente de ferramentas de desenvolvimento acessíveis, plataformas de hardware compatíveis e métodos eficientes de otimização de modelos que permitem o processamento local com baixo consumo energético.

1.4 ESTRUTURA DO TRABALHO

Este trabalho está organizado em sete capítulos. O Capítulo 1 apresenta o contexto, os objetivos, a hipótese e a organização geral do estudo. O Capítulo 2 desenvolve a revisão bibliográfica sobre TinyML e dispositivos embarcados, incluindo trabalhos relacionados e síntese crítica. O Capítulo 3 expõe a fundamentação teórica do fluxo de desenvolvimento TinyML (aquisição e preparação dos dados, escolha e treinamento de modelos, quantização, validação e implantação). O Capítulo 4 descreve o estudo de caso prático de classificação de resíduos sólidos, definindo escopo e etapas. O Capítulo 5 detalha a execução do fluxo (seleção do dataset, pré-processamento, arquitetura, treinamento, quantização, escolha de hardware e implementação). O Capítulo 6 apresenta e discute os resultados: métricas globais e por classe, matriz de confusão, curvas COR, desempenho computacional em dispositivo embarcado, comparação com estudo de referência e limitações com sugestões de melhoria. Por fim, o Capítulo 7 consolida as conclusões e indica perspectivas de continuidade.

2 REVISÃO BIBLIOGRÁFICA

2.1 TINYML E DISPOSITIVOS EMBARCADOS

TinyML é uma subárea do aprendizado de máquina dedicada à implementação de modelos de Aprendizado de Máquina em dispositivos embarcados com recursos computacionais limitados, como microcontroladores e sensores. Esses dispositivos, como o Arduino Nano 33 BLE Sense, Raspberry Pi, ESP32 e placas STM32, são amplamente utilizados em aplicações de IoT e automação, onde a eficiência energética e a baixa latência são fatores críticos. Os desafios técnicos incluem a limitação de memória, capacidade de processamento e consumo de energia desses dispositivos, o que exige técnicas específicas de otimização de modelos e algoritmos para garantir que as aplicações funcionem de maneira eficiente e eficaz.

2.2 TRABALHOS RELACIONADOS E ABORDAGENS NA LITERATURA

Diversos estudos têm demonstrado o potencial do TinyML em aplicações de diferentes áreas, especialmente aquelas que exigem soluções embarcadas com baixo consumo de energia e capacidade de processamento local. Os trabalhos apresentados a seguir utilizam técnicas, ferramentas e plataformas voltadas à implementação eficiente de modelos de aprendizado de máquina em dispositivos com recursos limitados.

Por exemplo, (WULNYE *et al.*, 2024) propõe uma solução aplicada à agricultura de precisão, utilizando imagens capturadas por um módulo de câmera VGA acoplada a um Arduino Nano 33 BLE Sense para detecção de avarias em folhas de milho. O modelo treinado possui tamanho de 321 kB e alcança uma acurácia (proporção de acertos do modelo) de 94,56%, demonstrando a viabilidade de aplicações visuais embarcadas com alto desempenho em campo. Ademais, o trabalho utilizou bancos de dados públicos de alta confiabilidade, como o repositório Harvard Dataverse (DATAVERSE, 2025) para treinamento do modelo aliado à plataforma Edge Impulse (EDGE IMPULSE, 2025b).

Outro exemplo é o estudo de (YAP; AHMAD, 2024), que apresenta uma solução simples para detecção de anomalias em um ventilador. Seu trabalho consiste em um giroscópio alinhado a um Arduino Nano 33 BLE Sense. O modelo de apenas 17 kB é capaz de detectar comportamentos anormais com 99,23% de acurácia. Para o treino do modelo, o autor realizou a coleta de dados empiricamente em dois cenários diferentes: um ventilador em funcionamento normal e outro com uma falha simulada com a quebra parcial de uma das hélices. O modelo foi treinado utilizando a plataforma LiteR, anteriormente conhecida como TensorFlow Lite (EDGE, 2024).

Dispositivos embarcados são em geral compactos e eficientes, podem realizar tarefas computacionais específicas por um custo de espaço e energia reduzidos. Neste sentido,

soluções TinyML podem ser ferramentas práticas para uso cotidiano, como mostra o estudo de (MELLIT *et al.*, 2025) utilizando sensor de infra-vermelho e um Arduino Portento H7 para detecção de danos em painéis fotovoltaicos, trabalhando na plataforma Edge Impulse e adquirindo os dados utilizando drone em fazendas algerianas, o modelo treinado alcançou uma acurácia de 98% e é apresentado no formato de pistola portátil com tela amigável para o usuário.

Nesses trabalhos citados, observa-se um fluxo de desenvolvimento recorrente para soluções baseadas em TinyML. As etapas principais de um projeto desse tipo podem ser resumidas da seguinte forma:

1. **Coleta de dados:** Captura de dados relevantes para o problema proposto, ou uso de repositórios públicos com conjuntos de dados apropriados;
2. **Pré-processamento:** Limpeza e preparação dos dados, incluindo normalização, remoção de ruídos, rotularização, seleção de características relevantes e aumento do número de dados utilizando IA (data augmentation), se necessário;
3. **Escolha e treinamento do modelo:** Uso de plataformas como TensorFlow ou Edge Impulse para treinar modelos de aprendizado de máquina com os dados já preparados;
4. **Quantização do modelo:** Redução do tamanho e da complexidade do modelo, tornando-o mais eficiente para execução em dispositivos embarcados com recursos limitados;
5. **Validação do modelo:** Avaliação do desempenho do modelo utilizando dados de validação, verificando sua precisão, robustez e capacidade de generalização;
6. **Ajuste fino:** Modificações em hiperparâmetros, arquitetura ou técnicas de regularização para aprimorar o desempenho do modelo;
7. **Implementação:** Integração do modelo quantizado no dispositivo embarcado, com testes em ambiente real e adaptação ao hardware utilizado.

A escolha do modelo e a quantização do modelo são peças-chaves na elaboração do projeto. Modelos de redes neurais como Redes Neurais Convolucionais (RNC) são frequentemente utilizados em aplicações de visão computacional, por exemplo. A quantização é particularmente importante em aplicações TinyML, pois trata-se de uma técnica que reduz a precisão dos pesos do modelo, permitindo que ele ocupe menos espaço e consuma menos recursos computacionais, sem comprometer significativamente o desempenho.

2.3 CONSIDERAÇÕES FINAIS DA REVISÃO

Durante a revisão bibliográfica foi observado que projetos TinyML podem ser executados em algumas etapas principais. Dentro de cada uma destas etapas decisões importantes são tomadas, como a escolha do modelo, a plataforma de desenvolvimento e a forma de coleta dos dados. As vastas opções de plataformas disponíveis como TensorFlow e Edge Impulse, bem como a variedade de dispositivos IoT, permitem uma vasta gama de soluções em Aprendizado de Máquina embarcado.

3 FUNDAMENTAÇÃO TEÓRICA

Neste capítulo, são abordados todas as etapas do fluxo de trabalho de desenvolvimento de soluções TinyML, explorando nuances e escolhas importantes a serem feitas em cada passo.

3.1 AQUISIÇÃO E PREPARAÇÃO DOS DADOS

O primeiro passo para o desenvolvimento de soluções TinyML é a coleta dos dados para treinamento. Esta etapa é fundamental, pois a quantidade e a qualidade dos dados influenciam diretamente a eficácia do modelo de aprendizado de máquina. Não há um número mínimo universal de dados necessários, nem um número mágico que funcione em todos os casos. Em geral, quanto mais representativos forem os dados e em maior volume estiverem disponíveis, melhor será o desempenho do modelo (ALTHNIAN *et al.*, 2021). Se a quantidade de dados não for suficiente, existem técnicas como *Aumento de Dados* que consiste em gerar novas amostras a partir dos dados existentes, aplicando transformações como rotação, escala, adição de ruído, entre outras (WHANG *et al.*, 2023).

Os métodos mais comuns de aquisição de dados podem ser divididos em duas grandes categorias: dados coletados empiricamente, realizando a captura de informação diretamente do ambiente ou realizando ensaios. Este método possui a vantagem de estar alinhado com a aplicação específica, o que leva a modelos mais eficazes (MILLER *et al.*, 2024), mas pode ser custoso e demorado. A outra categoria é a utilização de bancos de dados públicos, que podem ser encontrados em repositórios como o Kaggle (KAGGLE, 2023) ou Harvard Dataverse (DATAVERSE, 2025). Esses bancos de dados são frequentemente utilizados para treinamento de modelos e podem acelerar o processo de desenvolvimento, mas é importante garantir que os dados sejam representativos do problema específico a ser resolvido.

Após a coleta dos dados, é necessário realizar uma etapa de curadoria para remoção de ruídos, inconsistências, duplicatas e outras anomalias que podem afetar o treinamento do modelo. Ao limpar dados para treinamento de modelos, deve-se utilizar ferramentas próprias para aplicações em Aprendizado de Máquina (WHANG *et al.*, 2023). Quando aplicável, os dados também devem ser rotulados cuidadosamente, garantindo que cada amostra possua uma anotação precisa e consistente com o objetivo da tarefa de aprendizagem. Além disso, é necessário normalizar os dados, isto é, ajustar as escalas das amostras de forma que tenham tamanho, resolução, frequência e demais características semelhantes. Com isso, garantimos que o modelo não seja enviesado por dados com características muito distintas, aprendendo características irrelevantes (AHMED *et al.*, 2022).

3.2 DESENVOLVIMENTO E TREINAMENTO DE MODELOS

O desenvolvimento da solução começa com a escolha do tipo de modelo de aprendizado de máquina. Modelos de aprendizado de máquina são algoritmos que aprendem padrões a partir dos dados fornecidos, e sua escolha depende do tipo de problema a ser resolvido. Atualmente há uma variedade de modelos disponíveis, a Tabela 1 apresenta alguns dos modelos mais comuns utilizados em TinyML e suas principais funções (HAN; SIEBERT, 2022).

Tabela 1: Modelos de aprendizado de máquina e suas principais funções no contexto de TinyML.

Modelo	Função Principal
Redes Neurais Convolucionais (RNC)	Classificação de imagens
Redes Neurais Densas (RND)	Reconhecimento de palavras
Redes de Memória de Longo Prazo (RMLP)	Processamento de fala
Redes Neurais Recorrentes (RNR)	Processamento sequencial

Fonte: *Do autor*

Para o cumprimento do objetivo específico 4 foi utilizada uma Rede Neural Convolucional (RNC); a escolha concreta da arquitetura (MobileNetV2) é detalhada adiante no Capítulo 5, Seção 5.3.

Uma arquitetura de rede neural é construída basicamente por camadas de neurônios. Neurônios, neste contexto, são basicamente unidades de funções matemáticas que recebem valores de entradas, realiza uma soma ponderada, efetua uma operação matemática chamada de Função de Ativação e então direciona este resultado para a saída. Essa saída pode ser um neurônio de uma próxima camada ou a saída final do modelo. A Figura 1 mostra um simples exemplo.

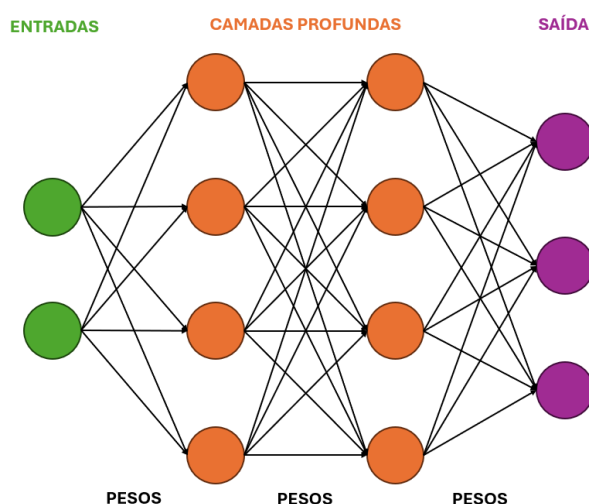


Figura 1: Exemplo de uma rede neural simples.

Fonte: *Do autor*

Os pesos exibidos na Figura 1 são os pesos da soma ponderada que são ajustados em cada neurônio durante o treinamento do modelo. Para realizar o ajuste destes pesos,

durante o treinamento é utilizado um algoritmo de otimização, este programa trata-se de uma técnica matemática que busca minimizar a diferença entre a saída prevista pelo modelo e a saída real dos dados de treinamento. Conforme descreve (POOJARY; PAI, 2019), o algoritmo mais comum utilizado para este fim é o *Gradiente Descendente Estocástico* (GDE), que ajusta os pesos da rede neural iterativamente, utilizando o gradiente da função de perda em relação aos pesos. Porém, ainda conforme (POOJARY; PAI, 2019), outros algoritmos como Adam e RMSprop também são amplamente utilizados.

Ademais, é importante considerar qual Função de Custo utilizar, pois é através dela que o algoritmo de otimização avalia o desempenho do modelo e determina como reajustar os pesos para melhorar o aprendizado. Há várias funções de custo que podem ser escolhidas para o treinamento do modelo. Esta escolha deve ser efetuada levando em consideração qual tipo de problema o aprendizado de máquina visa resolver (WANG *et al.*, 2022).

Tabela 2: Principais funções de perda para diferentes categorias de aprendizado de máquina.

Categoria	Funções de Perda
Problema de classificação	Perda 0-1, Perda Perceptron, Perda Logarítmica
Problema de regressão	Perda Quadrática, Perda Absoluta, Perda de Huber
Aprendizado não supervisionado	Erro Quadrático, Erro de Distância, Erro de Reconstrução

Fonte: Adaptado de (WANG *et al.*, 2022)

Na definição do treinamento da rede, existem alguns ajustes chamados de hiperparâmetros. Os hiperparâmetros incluem número de épocas, tamanho do lote e taxa de aprendizado. Cada época é definida como uma passagem completa por todos os dados de treinamento. O tamanho do lote é o número de amostras que o modelo processará antes de atualizar os pesos. A taxa de aprendizado é o quão rápido os pesos do modelo são atualizados.

Na prática, o treinamento de modelos de aprendizado de máquina é desenvolvido utilizando ferramentas próprias para este fim. Com base em (KALLIMANI *et al.*, 2024), podemos citar LiteRT, desenvolvido pela Google para dispositivos Android, iOS, Linux e microcontroladores, assim como μ Tensor desenvolvido pela ARM e Edge Impulse desenvolvido por Zach Shelby e Jan Jongboom.

3.3 QUANTIZAÇÃO PARA COMPACTAÇÃO DE MODELOS

A quantização é uma das técnicas mais críticas no ecossistema TinyML, atuando como o principal viabilizador da execução de redes neurais profundas em microcontroladores e dispositivos de borda. Conforme definem (JACOB *et al.*, 2018), a quantização consiste no processo de mapear valores de entrada de um conjunto grande (frequentemente contínuo, como números de ponto flutuante de 32 bits – FP32) para um conjunto menor de valores discretos (como inteiros de 8 bits – INT8).

Essa transformação não visa apenas a redução do tamanho do arquivo do modelo armazenado em memória Flash, mas também a redução da latência de inferência e do consumo energético, uma vez que operações aritméticas com inteiros de 8 bits são computacionalmente menos custosas que operações em ponto flutuante (GHOLAMI *et al.*, 2021).

3.3.1 Formulação Matemática

A abordagem mais comum em frameworks como TensorFlow Lite é a *Quantização Uniforme Afim*. Neste esquema, um valor real r (ponto flutuante) é mapeado para um valor quantizado inteiro q através de dois parâmetros principais: uma constante de escala S (*scale*) e um ponto zero Z (*zero-point*). A relação é dada pela Equação 1:

$$q = \text{clamp} \left(\text{round} \left(\frac{r}{S} \right) + Z \right) \quad (1)$$

Onde:

- r é o valor real (peso ou ativação original);
- S é um fator de escala (float32) que determina o tamanho do passo entre valores discretos;
- Z é o ponto zero (inteiro), que permite que o valor real 0.0 seja representado exatamente por um valor inteiro quantizado, garantindo eficiência em operações de *padding* (preenchimento);
- $\text{round}(\cdot)$ é a operação de arredondamento para o inteiro mais próximo;
- $\text{clamp}(\cdot)$ limita o valor ao intervalo suportado (ex: $[0, 255]$ para uint8 ou $[-128, 127]$ para int8).

Para recuperar a aproximação do valor real durante a inferência (ou desquantização), utiliza-se a Equação 2:

$$r \approx S \cdot (q - Z) \quad (2)$$

3.3.2 Tipos de Quantização

Existem diferentes estratégias para aplicar a quantização, sendo as duas principais:

1. **Quantização Pós-Treinamento (PTQ - *Post-Training Quantization*)**: Aplica-se a quantização após o modelo ter sido completamente treinado em ponto flutuante. É o método mais rápido e acessível, pois não requer retreinamento. Geralmente, utiliza-se um pequeno conjunto de dados representativo para calibrar os valores de S e Z , minimizando a perda de acurácia. Esta foi a abordagem selecionada para o estudo de caso deste trabalho (detalhada no Capítulo 5).
2. **Treinamento Ciente de Quantização (QAT - *Quantization-Aware Training*)**: Simula os efeitos da quantização (ruído e arredondamento) durante a fase de treinamento (forward pass), permitindo que a rede neural aprenda parâmetros mais robustos à perda de precisão. Embora resulte em maior acurácia, possui um custo computacional e complexidade de implementação mais elevados.

3.3.3 Impacto na Acurácia e Desempenho

Embora a quantização ofereça ganhos significativos de eficiência – frequentemente reduzindo o uso de memória em 4x (de 32 bits para 8 bits) – ela introduz um erro inerente, conhecido como ruído de quantização. A eficácia da técnica depende da robustez da arquitetura da rede neural em tolerar essa redução de precisão sem degradar drasticamente a acurácia da classificação final. No contexto deste trabalho, a escolha pela quantização INT8 busca o equilíbrio ideal entre a viabilidade de execução no Raspberry Pi e a manutenção da capacidade preditiva do modelo MobileNetV2.

3.4 VALIDAÇÃO, AJUSTE FINO E MÉTRICAS DE DESEMPENHO

A validação dos modelos de aprendizado de máquina consiste em avaliar se o modelo está desempenhando adequadamente sua função. Como apresentado na Seção 2.2, uma prática comum é reservar de 20% a 30% dos dados coletados para testes, após o treinamento. Com base nesses testes, são calculadas métricas de desempenho. As principais métricas para avaliação de classificadores binários, segundo (RAINIO; TEUHO; KLÉN, 2024), são: **acurácia** (ACU), **sensibilidade** (SEN), **especificidade** (ESP), **precisão** (PRE) e **medida F1** (F1), definidas nas Equações 3, 4, 5, 6 e 7, respectivamente.

$$ACU = \frac{VP + VN}{VP + VN + FP + FN} \in [0, 1], \quad (3)$$

$$SEN = Rec. = \frac{VP}{VP + FN} \in [0, 1], \quad (4)$$

$$ESP = \frac{VN}{VN + FP} \in [0, 1], \quad (5)$$

$$PRE = \frac{VP}{VP + FP} \in [0, 1]. \quad (6)$$

A medida F1 é definida como a média harmônica entre a precisão e a sensibilidade.

$$F1 = 2 \cdot \frac{PRE \cdot SEN}{PRE + SEN} \quad (7)$$

Onde:

- **VP**: Verdadeiros Positivos, número de instâncias corretamente classificadas como positivas.
- **VN**: Verdadeiros Negativos, número de instâncias corretamente classificadas como negativas.
- **FP**: Falsos Positivos, número de instâncias incorretamente classificadas como positivas.
- **FN**: Falsos Negativos, número de instâncias incorretamente classificadas como negativas.

Ademais, existem algumas métricas com apelo mais visual, como é o caso do gráfico denominado Característica de Operação do Receptor (COR). Um gráfico COR é obtido a partir da variação do Limiar de Decisão, calculando especificidade e sensibilidade e então plotando num gráfico. A Figura 2 exibe um exemplo de curva COR, um comparativo entre o desempenho de dois modelos.

Um gráfico COR de um modelo ideal seria composto por uma reta vertical de (1,0) até (1,1) e então uma reta horizontal de (1,1) até (0,1), indicando máxima sensibilidade e especificidade. Ainda sobre o gráfico COR, uma métrica para desempenho é a Área Sobre a Curva (ASC). No caso ideal, $ASC = 1$. Desta forma, quanto mais tender ao ponto (1,1) o gráfico COR, melhor o modelo. Da mesma forma, quanto mais elevado o indicador ASC, melhor.

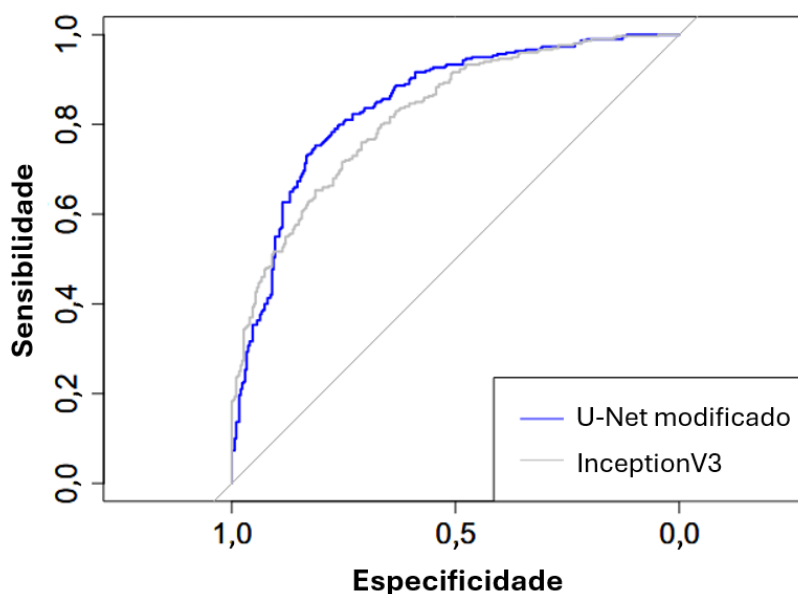


Figura 2: Exemplo de um gráfico COR.

Fonte: Adaptado de (RAINIO; TEUHO; KLÉN, 2024)

Ainda sobre métricas visuais, temos a Matriz de Confusão. Esta matriz relaciona os valores previstos pelo modelo com os valores reais. A Figura 3 mostra um exemplo deste tipo de métrica.

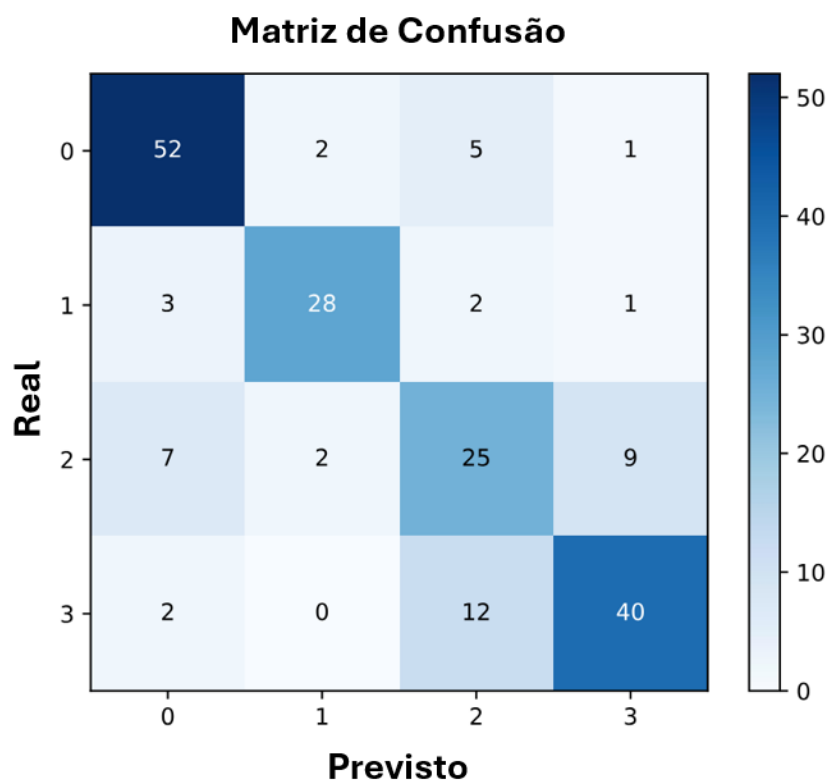


Figura 3: Exemplo de uma Matriz de Confusão.

Fonte: Adaptado de (ALSHAMMARI, 2024)

No caso de um modelo ideal, todos os valores da matriz de confusão estariam na diagonal principal da matriz, indicando que o modelo acerta 100% das predições.

Finalmente, caso as métricas de desempenho do modelo não sejam satisfatórias, realizam-se ajustes nos hiperparâmetros para melhorar sua resposta; esse processo, denominado Ajuste Fino, foi executado e está detalhado no Capítulo 5, Seção 5.3.

3.5 IMPLANTAÇÃO EM DISPOSITIVOS EMBARCADOS

Há vários dispositivos no mercado que podem trabalhar com soluções TinyML. (OLIVEIRA *et al.*, 2024) realiza uma pesquisa dos principais dispositivos no mercado comparando suas configurações. Estas comparações são exibidas na Tabela 3.

Tabela 3: Especificações de dispositivos TinyML de baixo e alto desempenho.

Categoria	Placa	Custo (US\$)	Corrente (A)	RAM (MB)	Arquitetura	Freq. (MHz)
Low-end	Arduino Nano BLE 33	~25	0.5	0.256	32-bit	64
	Adafruit EdgeBadge	~36	1.0	0.192	32-bit	120
	ESP32 DevKitC	~10	0.5	0.512	32-bit	240
	Raspberry Pi Pico W	~7	0.5	0.256	32-bit	133
	STM32F746	~15	0.5	0.5	32-bit	216
	Sipeed Maix Bit	~45	1.0	8.0	64-bit	600
	SparkFun Edge	~17	0.5	0.384	32-bit	96
High-end	Banana Pi M2 Zero	~20	2.0	512	32-bit	1200
	Orange Pi Zero 3	~35	2.0	1000	64-bit	1500
	Raspberry Pi Zero W	~15	1.2	512	32-bit	1000
	Raspberry Pi Zero 2 W	~20	1.2	512	64-bit	1000
	Raspberry Pi 3 model B	~35	2.5	1000	64-bit	1200
	Raspberry Pi 4 model B	~35	3.0	4000	64-bit	1800
	Raspberry Pi 5	~56	5.0	4000	64-bit	2400
	NVIDIA Jetson Nano	~99	5.0	4000	64-bit	1430

Fonte: Adaptado de (OLIVEIRA *et al.*, 2024)

É fundamental que a plataforma escolhida e o modelo desenvolvido sejam compatíveis, ou seja, suas configurações e requisitos devem ser alinhados para evitar desperdício de recursos computacionais e garantir uma solução final eficiente. Além disso, é essencial considerar o consumo de energia e garantir que a escolha atenda às restrições do projeto.

A implantação do modelo final varia conforme a plataforma de desenvolvimento escolhida. Principais ferramentas como TensorFlow e Edge Impulse permitem gerar bibliotecas contendo o modelo treinado quantizado completo, prontas para integração no código do dispositivo embarcado, conforme descrito em (EDGE IMPULSE, 2025a) e (TENSORFLOW, 2023). Ambas oferecem suporte a diversas linguagens de programação, como C++ e Python, garantindo flexibilidade no desenvolvimento e na integração com diferentes tipos de hardware.

Um comparativo entre as ferramentas TensorFlow e Edge Impulse pode ser visto em (ESSANOAH ARTHUR *et al.*, 2024), que compara o desempenho de dois modelos que servem ao mesmo propósito, que neste caso é identificação de doenças em folhas de milho.

As Tabelas 4 e 5 mostram uma comparação entre os consumos de recursos computacionais e uma comparação das métricas de desempenho, respectivamente.

Tabela 4: Comparação de desempenho entre TensorFlow e Edge Impulse.

Métrica	TensorFlow	Edge Impulse
RAM	890,5 kB	726,6 kB
Flash	374,4 kB	344,7 kB
Latência	84,99 ms	76,48 ms

Fonte: Adaptado de (ESSANOAH ARTHUR *et al.*, 2024)

Tabela 5: Comparação de métricas de desempenho entre TensorFlow e Edge Impulse.

Métrica	TensorFlow	Edge Impulse
Acurácia no treinamento	97%	96%
Acurácia na validação	96,54%	95%
Acurácia no teste	96,38%	94,84%

Fonte: Adaptado de (ESSANOAH ARTHUR *et al.*, 2024)

4 ESTUDO DE CASO: CLASSIFICAÇÃO DE RESÍDUOS SÓLIDOS

Neste capítulo é apresentado o estudo de caso realizado, baseado nos conceitos e técnicas discutidos nos capítulos anteriores. O objetivo é implementar e validar, na prática, uma solução de aprendizado de máquina embarcado seguindo o fluxo de trabalho proposto.

A escolha pela classificação de resíduos se justifica pela necessidade de automatizar uma etapa que, segundo a fonte (PLÁSTICOS, 2025), ainda é realizada manualmente e envolve riscos aos trabalhadores. Essa atividade expõe os operadores a perigos físicos e ambientais, tornando relevante o desenvolvimento de soluções que reduzam esse contato direto.



Figura 4: *Processo de seleção manual de resíduos.*

Fonte: Fonte: (PLÁSTICOS, 2025)

4.1 DESCRIÇÃO DO ESTUDO DE CASO

Em continuidade aos objetivos traçados neste trabalho, o presente estudo de caso aplica, na prática, o fluxo de desenvolvimento TinyML para resolver o problema de classificação de resíduos sólidos. O projeto busca consolidar os conhecimentos adquiridos ao longo do trabalho, conectando teoria e prática, e evidenciar, de forma aplicada, as vantagens do aprendizado de máquina embarcado. Além disso, foi realizada uma comparação entre a solução TinyML desenvolvida e abordagens tradicionais de aprendizado de máquina, de

modo a destacar os diferenciais e limitações de cada abordagem no contexto do problema proposto.

O estudo de caso consiste no desenvolvimento de um modelo de aprendizado de máquina embarcado para categorização de lixo em 12 categorias diferentes:

- Pilhas e baterias;
- Orgânico;
- Vidro marrom;
- Vidro transparente;
- Vidro verde;
- Papelão;
- Roupas;
- Metal;
- Papel;
- Plástico;
- Calçados;
- Rejeitos.

Este estudo de caso foi definido por tratar-se de um tema altamente relevante para o meio ambiente e pela ampla disponibilidade de dados em repositórios públicos, o que facilita a condução do fluxo de trabalho. O modelo foi treinado utilizando a plataforma TensorFlow, visto que se trata de uma solução gratuita e de código aberto, além de apresentar resultados superiores de acurácia e desempenho conforme discutido na Seção 3.5. A coleta dos dados foi baseada no conjunto de dados *RealWaste* (SINGLE; IRANMANESH; RAAD, 2023a), disponibilizado pelo UCI Machine Learning Repository (UCI, 2024). A escolha por este conjunto justifica-se por sua associação a um estudo de referência publicado, o que possibilita a realização de comparações diretas de desempenho entre a abordagem TinyML proposta e classificadores de estado da arte — detalhadas posteriormente na Seção 5.1.

4.2 ETAPAS DO ESTUDO DE CASO

O estudo de caso foi desenvolvido em etapas, baseada no fluxo de trabalho apresentado na Seção 2.2 com a adição de alguns pontos. As etapas são apresentadas a seguir com suas respectivas atribuições:

1. **Coleta de dados:** A primeira etapa consiste em obter imagens de lixo categorizadas a partir de repositórios públicos. Essa abordagem garante acesso a dados variados e já organizados em classes, essenciais para o treinamento de modelos de aprendizado de máquina. A diversidade do conjunto de dados é crucial para que o modelo possa generalizar bem a diferentes tipos de resíduos.

2. **Pré-processamento:** Após a coleta, as imagens passam por uma etapa de limpeza e normalização, que inclui o redimensionamento para tamanhos uniformes e o aumento de dados (*data augmentation*). Essas técnicas visam melhorar a qualidade do conjunto de dados, aumentar sua robustez e reduzir possíveis vieses durante o treinamento.
3. **Desenvolvimento e treinamento do modelo:** Nesta etapa, foi treinada uma Rede Neural Convolutiva (RNC) utilizando TensorFlow. Essa arquitetura é ideal para classificação de imagens, permitindo que o modelo aprenda padrões visuais complexos necessários para identificar diferentes categorias de lixo.
4. **Escolha da plataforma de hardware:** O hardware ideal para execução do modelo foi avaliado com base na complexidade do programa e nos requisitos de desempenho, consumo energético e custo. Essa etapa garante que o dispositivo escolhido seja compatível com as restrições de um sistema embarcado.
5. **Quantização do modelo:** Técnicas de quantização foram aplicadas para reduzir o tamanho e os requisitos computacionais do modelo, permitindo sua execução eficiente em dispositivos embarcados. Essa etapa é fundamental para garantir a viabilidade da solução TinyML.
6. **Validação e ajuste fino:** O modelo treinado foi avaliado utilizando métricas como acurácia, precisão e *F1-score*. Com base nos resultados obtidos, ajustes foram realizados para melhorar o desempenho do modelo, corrigindo possíveis falhas de generalização ou subaproveitamento.
7. **Implantação:** Após a validação, o modelo quantizado foi integrado em um micro-controlador compatível. Testes práticos foram realizados para avaliar o desempenho da solução em um ambiente real, verificando sua eficiência e tempo de resposta.
8. **Avaliação dos resultados:** Nesta etapa, foram realizados comparativos entre as métricas de desempenho elencadas na Seção 3.4 da solução TinyML desenvolvida e aplicações de inteligência artificial tradicionais. O objetivo é destacar as vantagens em termos de eficiência, latência, custo e desempenho.
9. **Escrita do Trabalho de Conclusão II:** A escrita do relatório foi uma atividade contínua, desenvolvida em paralelo às demais etapas do cronograma. Isso assegura que todos os passos sejam documentados detalhadamente, facilitando a organização e o cumprimento dos prazos acadêmicos.

Demais detalhes de cada etapa foram discutidos e decididos durante o desenvolvimento do trabalho, considerando as especificidades do modelo e do dispositivo embarcado escolhido.

5 DESENVOLVIMENTO

Este capítulo apresenta a execução prática do fluxo de trabalho proposto no TCC-I, abrangendo todas as etapas do desenvolvimento da solução TinyML para classificação de resíduos sólidos. São detalhados o processo de seleção e preparação do conjunto de dados, a implementação e o treinamento do modelo em ambiente de nuvem, as técnicas de quantização aplicadas e a implantação da rede neural em um dispositivo embarcado.

Ao longo deste capítulo e dos próximos, foram citados trechos de código que podem ser consultados pelo leitor em: github.com/maxwellrs-dev/ai4rpi.

5.1 COLETA E SELEÇÃO DO BANCO DE DADOS

Inicialmente, havia sido planejada a utilização de um conjunto de dados público do *Kaggle* (KAGGLE, 2023), contendo 12 categorias distintas de resíduos sólidos. No entanto, optou-se por empregar o conjunto de dados *RealWaste* (SINGLE; IRANMANESH; RAAD, 2023a), proveniente do UCI Machine Learning Repository, em virtude de estar associado ao artigo *A Novel Real-Life Data Set for Landfill Waste Classification Using Deep Learning* (SINGLE; IRANMANESH; RAAD, 2023b). Essa associação permite a realização de comparações diretas entre o modelo desenvolvido neste trabalho e o apresentado no estudo de referência, fortalecendo a análise dos resultados.

O conjunto *RealWaste* é composto por imagens reais de resíduos coletados em aterros sanitários, classificadas conforme o tipo de material predominante. Os rótulos atribuídos às imagens representam a categoria principal. A Tabela 6 apresenta as categorias e a quantidade de amostras disponíveis em cada uma delas.

Tabela 6: Categorias e quantidade de imagens no conjunto de dados *RealWaste*.

Categoria	Quantidade de imagens
Papelão	461
Resíduos orgânicos alimentares	411
Vidro	420
Metal	790
Lixo diverso	495
Papel	500
Plástico	921
Têxteis	318
Vegetação	436
Total	4.752

Fonte: Adaptado de (SINGLE; IRANMANESH; RAAD, 2023a)

O total de 4.752 imagens fornece um conjunto de dados de tamanho moderado, adequado para o treinamento de redes neurais convolucionais compactas voltadas a aplicações embarcadas. O dataset apresenta boa diversidade de materiais, o que contribui para a generalização do modelo em cenários reais.

A Figura 5 apresenta exemplos de imagens pertencentes ao conjunto de dados *RealWaste*, ilustrando algumas das categorias utilizadas no treinamento do modelo. As amostras demonstram a variabilidade visual entre diferentes tipos de materiais, evidenciando as diferenças de textura, cor e formato que o modelo deve aprender a distinguir durante o processo de classificação.

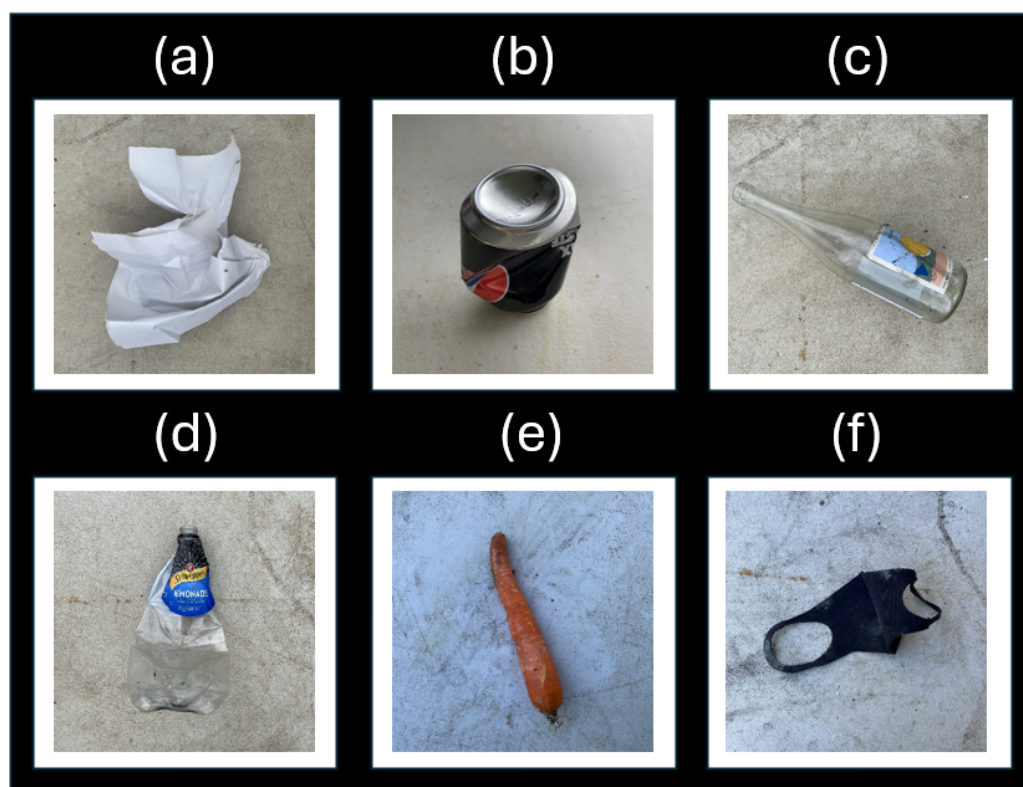


Figura 5: Exemplos de imagens do conjunto de dados *RealWaste*: (a) papel; (b) metal; (c) vidro; (d) plástico; (e) resíduo orgânico alimentar; (f) resíduo têxtil.

Fonte: Adaptado de (SINGLE; IRANMANESH; RAAD, 2023a)

5.2 PRÉ-PROCESSAMENTO DOS DADOS

O conjunto de dados *RealWaste* já se apresenta homogêneo quanto à resolução (522×522 px) e à organização em subpastas por classe, o que dispensou etapas extensas de limpeza, remoção de ruído ou correções manuais. Assim, adotou-se um pré-processamento propositalmente simples, com dois objetivos principais: (i) adequar o tamanho das imagens à entrada da rede e (ii) balancear as classes para reduzir vieses no treinamento.

As etapas executadas foram:

1. **Redimensionamento para a entrada do modelo.** Durante o carregamento, cada imagem foi redimensionada para 224×224 px, tamanho compatível com arquiteturas convolucionais compactas e com as restrições de memória/processamento

do alvo embarcado. Tal ajuste é executado diretamente pela rotina de processamento (`metadados/aumento_de_dados.py`) ao invocar a função `load_img(..., target_size=(224,224))`, assegurando que todo o conjunto balanceado seja gravado já no tamanho esperado pelo modelo.

2. **Balanceamento via aumento de dados (data augmentation).** Após contabilizar as imagens por classe, definiu-se uma meta por classe igual ao maior valor entre a classe mais populosa do conjunto. Na prática, isso faz com que todas as classes sejam expandidas até o tamanho da classe majoritária, que neste caso é de 921. Para gerar amostras sintéticas preservando a semântica, aplicaram-se apenas transformações geométricas e fotométricas leves com *ImageDataGenerator*: rotação ($\pm 20^\circ$), deslocamentos horizontais e verticais (até 20%), cisalhamento (0,15), *zoom* (0,15), espelhamento horizontal e variação de brilho entre 0,7 e 1,3. O espelhamento vertical foi evitado por potencialmente alterar a plausibilidade física de algumas cenas. Limitou-se a no máximo 10 variações por imagem base, reduzindo redundância. As imagens originais redimensionadas foram copiadas para o diretório de saída e, em seguida, as variações foram geradas até atingir a meta por classe.

Todo o procedimento está implementado em `metadados/aumento_de_dados.py`, que preserva a organização hierárquica original do dataset (com subpastas individuais para cada categoria), grava o conjunto balanceado em disco e facilita a reprodução dos resultados.

5.3 DESENVOLVIMENTO E TREINAMENTO DO MODELO

Esta seção descreve a implementação prática do modelo de classificação, desde a preparação do fluxo de dados (*pipeline*) até o treinamento com ajuste fino. A implementação foi conduzida em ambiente de nuvem (Google Colab) utilizando TensorFlow/Keras. Essa escolha apoia-se na facilidade de acesso (basta um navegador), disponibilidade gratuita — ainda que limitada — de CPU/GPU/TPU e integração direta com Google Drive para armazenamento e compartilhamento de artefatos.

5.3.1 Fluxo de dados

Os conjuntos de treino e validação foram criados a partir do diretório base do *RealWaste*, empregando TensorFlow, com divisão estratificada aproximada de 80/20 e semente 123 para reprodutibilidade. As imagens foram redimensionadas para 224×224 px e carregadas em **lotes** de 64 amostras.

5.3.2 Arquitetura e estratégia de treinamento

Utilizou-se a **MobileNetV2** como extratora de atributos base (`include_top = False`, `weights = 'imagenet'`, `alpha = 1`), por ser uma arquitetura leve, adequada ao contexto TinyML. Sobre essa base adicionaram-se camadas finais para classificação multi-classe:

- `GlobalAveragePooling2D`;
- `Dropout(0,2)` para regularização;
- `Dense(128, ReLU)`;
- `Dense(#classes, Softmax)`.

A estratégia de treinamento adotou duas fases principais:

1. **Pré-treino**: base congelada (`base_model.trainable = False`) por 5 épocas, para estabilizar as camadas adicionadas;
2. **Ajuste fino**: base liberada parcialmente, mantendo congeladas todas as camadas exceto as **80 últimas**, permitindo especialização progressiva dos filtros superiores à tarefa.

5.3.3 Função de perda, otimizador e mecanismos de controle (callbacks)

O modelo foi compilado com `loss = 'sparse_categorical_crossentropy'` e métrica principal foi a acurácia. Como otimizador empregou-se **AdamW** com taxa de aprendizado inicial de 1×10^{-4} e *decaimento de peso* (*weight decay*) de 1×10^{-5} .

Para maior robustez frente a eventuais desequilíbrios residuais, utilizaram-se **pesos por classe** obtidos via `sklearn.utils.compute_class_weight` sobre os **rótulos** do conjunto de treino.

Dois mecanismos de controle (*callbacks*) acompanharam o processo de otimização:

- **Parada antecipada** (*EarlyStopping*) monitorando `val_loss`, com `patience = 10` e restauração dos melhores pesos;
- **Redução da taxa em platô** (*ReduceLROnPlateau*) com `factor = 0,5`, `patience = 3`, `min_lr = 1e-6`, reduzindo a taxa de aprendizado quando a validação estagna.

O treinamento completo foi limitado a **150 épocas** e interrompido pela parada antecipada quando apropriado.

5.3.4 Resultados do treino

Durante o treinamento, foram registradas as curvas de **acurácia** e **perda** para treino e validação, respectivamente. A Figura 6 ilustra a evolução dessas métricas e o ponto de parada determinado pela parada antecipada.

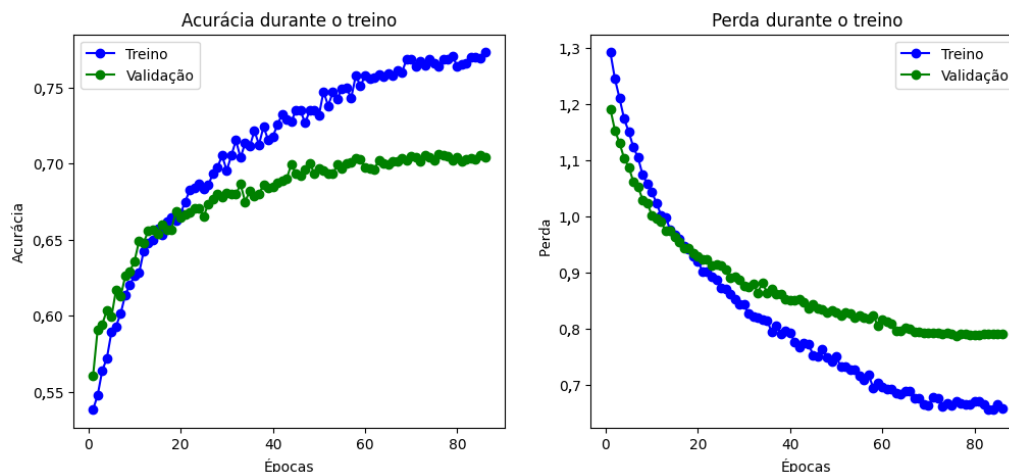


Figura 6: Curvas de acurácia e perda por época (treino e validação).

Fonte: Do autor

Ao final, o modelo em ponto flutuante foi salvo para posterior conversão e quantização (Seção 5.4).

5.4 QUANTIZAÇÃO DO MODELO

Nesta etapa, o modelo treinado em ponto flutuante foi convertido para o formato TensorFlow Lite com **quantização pós-treinamento** para **INT8**, visando reduzir tamanho do arquivo do modelo, uso de memória e latência de inferência, tornando-o adequado a dispositivos embarcados com recursos limitados.

5.4.1 Abordagem adotada

Empregou-se **quantização estática** com calibração via *conjunto representativo*. Os passos implementados no código Python foram:

1. Criar o conversor:

```
converter = tf.lite.TFLiteConverter.from_keras_model(model)
```

2. Ativar otimizações:

```
converter.optimizations = [tf.lite.Optimize.DEFAULT]
```

3. Definir gerador representativo (100 lotes):

```
def representative_data_gen():
    for _ in range(100):
        imgs, _ = next(train_ds_iter)
        yield [tf.cast(imgs, tf.float32)]
converter.representative_dataset = representative_data_gen
```

4. Restringir operações a inteiros:

```
converter.target_spec.supported_ops =
    [tf.lite.OpsSet.TFLITE_BUILTINS_INT8]
```

5. Fixar tipos de entrada/saída:

```
converter.inference_input_type = tf.uint8
converter.inference_output_type = tf.uint8
```

6. Converter e salvar:

```
tflite_model = converter.convert()
with open(tflite_model_path, "wb") as f:
    f.write(tflite_model)
```

5.4.2 Validação do modelo quantizado

O arquivo quantizado foi carregado via `tf.lite.Interpreter`. Para a avaliação, os tensores foram alocados e os metadados de entrada/saída inspecionados; as imagens foram convertidas condicionalmente para `uint8` conforme o tipo requerido pelo tensor de entrada; em seguida, realizou-se a inferência com `invoke()` e coletaram-se as distribuições de probabilidades por classe. A partir dessas pontuações, foram calculadas a acurácia, a precisão ponderada, a sensibilidade ponderada, o F1 ponderado, além da matriz de confusão normalizada e das curvas COR em esquema multiclasse.

Em síntese, a quantização INT8 permitiu obter um modelo compacto e eficiente sem alterar o fluxo principal de treinamento, viabilizando sua execução futura em dispositivos embarcados conforme discutido nas próximas seções.

5.5 ESCOLHA E PREPARAÇÃO DO HARDWARE

Nesta seção é apresentada a justificativa da escolha da plataforma de execução do modelo e a preparação realizada para torná-la apta à inferência embarcada. A escolha pela Raspberry Pi 3 Model B+ foi definida com base em um conjunto de critérios técnicos que a qualificam como uma plataforma ideal para a validação de soluções TinyML em ambiente de borda, oferecendo o equilíbrio ideal entre capacidade e restrição.

Os principais fatores considerados foram: o equilíbrio entre recursos computacionais (CPU quad-core 64-bit e 1 GB de RAM) e simplicidade de uso; a compatibilidade nativa com o fluxo de quantização INT8 (uso do `tflite_runtime`) sem a necessidade de aceleração especializada; o suporte direto a câmera para aquisição das imagens de teste; e a ampla comunidade e maturidade do ecossistema (documentação consolidada, suporte direto a Python e TensorFlow Lite).

Esses fatores tornam a placa adequada como alvo intermediário: suficientemente capaz para validar latência e robustez do modelo convolucional quantizado, mas ainda representativa de um ambiente de recursos restritos frente a soluções de maior porte. Adicionalmente, a disponibilidade física imediata da plataforma no início do projeto permitiu eliminar custos e atrasos na aquisição do hardware.

A Tabela 7 resume as principais especificações técnicas empregadas no protótipo.

Tabela 7: *Especificações principais da Raspberry Pi 3 Model B+.*

Característica	Especificação
Processador	Broadcom BCM2837B0, Cortex-A53 (ARMv8) 64-bit, 1,4 GHz
Memória RAM	1 GB LPDDR2 SDRAM
Conectividade	Wi-Fi 2,4/5 GHz 802.11 b/g/n/ac; Bluetooth 4.2 BLE
Portas USB	4 USB 2.0
GPIO	40 pinos (compatível com versões anteriores)
Saída de vídeo	1 porta HDMI
Periféricos	Conectores DSI (display) e CSI (câmera)
Áudio/Vídeo analógico	Conector P2 4 polos (áudio estéreo + vídeo composto)
Armazenamento	Slot microSD (SO e dados)
Alimentação	5 V / 2,5 A (mínimo) via micro USB

Fonte: Adaptado de (FOUNDATION, 2025).

5.5.1 Preparação e configuração inicial

Os passos principais para preparar o dispositivo para a execução do modelo foram:

1. **Gravação do sistema operacional:** uso do Raspberry Pi Imager para instalar Raspberry Pi OS Lite (versão 64-bit), disponível no site oficial (FOUNDATION, 2025);
2. **Acesso remoto:** habilitação de SSH e definição de hostname específico para identificação na rede local;
3. **Atualização de pacotes:** execução de atualização e cuidado em manter versão de Python compatível com dependências de inferência (TensorFlow Lite requer armv8 64-bit);
4. **Instalação das dependências:** instalação de `python3-pip`, bibliotecas de suporte (`numpy`, `tf-lite-runtime` ou `tensorflow-lite`);
5. **Habilitação de periféricos:** ativação opcional do suporte à câmera via `raspi-config`;
6. **Validação do ambiente:** teste de carregamento do modelo quantizado, verificação de memória disponível, tempo médio de inferência e temperatura sob carga leve.

5.6 IMPLEMENTAÇÃO E DEMONSTRAÇÃO PRÁTICA

Esta subseção descreve a execução prática do modelo quantizado em uma Raspberry Pi 3 Model B+, conforme especificado na Seção 5.5. O objetivo é demonstrar a captura de imagem, pré-processamento mínimo e inferência usando o `tf-lite-runtime`, validando o fluxo ponta-a-ponta em ambiente embarcado.

O código na sua íntegra está disponível no link do GitHub mencionado inicialmente, no arquivo `metadados/test_inynml_pi.py`.

5.6.1 Fluxo de inferência

Assumem-se as seguintes bibliotecas importadas e o caminho do modelo definidos previamente:

```
import cv2, time; import numpy as np
from tf.lite_runtime.interpreter import Interpreter
MODEL_PATH = "mobilenet_test.tflite"
```

Os passos principais executados pela rotina são:

1. Captura de um quadro da câmera.

```
cap = cv2.VideoCapture(0); ret, frame = cap.read(); cap.release()
```

2. Armazenamento da imagem original para registro e auditoria (foto_teste.jpg).

```
cv2.imwrite("foto_teste.jpg", frame)
```

3. Redimensionamento para 224×224 px (entrada do modelo).¹

```
rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB); img = cv2.resize(rgb, (224,
224), interpolation=cv2.INTER_AREA)
```

4. Expansão da dimensão de lote (shape (1,224,224,3)).

```
input_data = np.expand_dims(img, axis=0).astype(np.uint8)
```

5. Carregamento do arquivo .tflite, alocação de tensores e leitura dos descritores de I/O.

```
interpreter = Interpreter(model_path=MODEL_PATH);
interpreter.allocate_tensors(); inp =
interpreter.get_input_details()[0]; out =
interpreter.get_output_details()[0]
```

6. Escrita dos dados de entrada e invocação da inferência (com medição simples de latência).

```
interpreter.set_tensor(inp['index'], input_data); start = time.time();
interpreter.invoke(); latency_ms = (time.time() - start) * 1000
```

7. Leitura das probabilidades de saída e seleção da classe de maior valor (*argmax*).

```
output = interpreter.get_tensor(out['index'])[0]; pred_index =
int(np.argmax(output))
```

¹MobileNetV2 tradicionalmente utiliza normalização para [0,1] ou centragem em [-1,1]; o modelo quantizado foi calibrado diretamente em uint8, permitindo uso direto do frame bruto.

6 RESULTADOS E DISCUSSÃO

Neste capítulo, apresentamos e discutimos os resultados experimentais do estudo, analisando o desempenho do modelo nas principais métricas, a eficiência computacional da inferência em dispositivo embarcado e as limitações e implicações práticas dos achados à luz dos objetivos e da literatura.

6.1 AVALIAÇÃO DE DESEMPENHO

Esta seção sumariza o desempenho do classificador com base no roteiro executado no Google Colab, avaliando o modelo quantizado em INT8 sobre o conjunto de validação. São reportadas métricas globais (médias ponderadas), a matriz de confusão normalizada por classe e as curvas COR multiclasse.

6.1.1 Métricas globais

As métricas foram computadas via `scikit-learn` diretamente sobre as predições do intérprete TFLite:

- **Acurácia** (ACU): proporção de acertos no conjunto de validação;
- **Precisão ponderada** (PRE): média ponderada da precisão por classe, ponderada pelo suporte (número de amostras por classe);
- **Sensibilidade ponderada** (SEN): média ponderada da sensibilidade por classe, ponderada pelo suporte;
- **F1 ponderado**: média ponderada do F1 por classe, combinando precisão e sensibilidade.

Os resultados consolidados para o conjunto de validação são os exibidos na Tabela 8.

Tabela 8: Métricas globais (validação) — modelo TFLite INT8.

Métrica	Valor
Acurácia	0,68
Precisão (ponderada)	0,69
Sensibilidade (ponderada)	0,68
F1-score (ponderado)	0,68

6.1.2 Comparação com o estudo de referência

O presente trabalho utilizou o mesmo conjunto de dados *RealWaste* empregado no estudo de referência (SINGLE; IRANMANESH; RAAD, 2023c), o que permite estabelecer uma linha de base comum para análise. No entanto, é fundamental destacar que a comparação direta entre os resultados de acurácia absoluta deve ser feita com ressalvas, uma vez que os objetivos e as restrições de cada abordagem são fundamentalmente distintos.

O estudo de referência teve como propósito validar a qualidade do conjunto de dados e estabelecer marcos de desempenho (*benchmarks*) utilizando o estado da arte em Redes Neurais Convolucionais. Para isso, foram empregadas arquiteturas profundas e computacionalmente intensivas (como InceptionV3 e DenseNet121), imagens em alta resolução (524×524 px) e técnicas de aumento de dados (*data augmentation*) robustas e agressivas. Nesse cenário ideal, livre de restrições de hardware, o estudo reportou acurácias superiores a 89%.

Em contrapartida, o objetivo deste trabalho foi validar a viabilidade de uma aplicação *TinyML* em um dispositivo de borda (Raspberry Pi 3). Para atender aos requisitos de latência em tempo real e limitações de memória (RAM e Flash) descritos na Seção 6.2, foram necessárias escolhas de projeto que penalizam intencionalmente a capacidade preditiva em favor da eficiência:

- **Arquitetura:** Adoção da MobileNetV2, uma rede significativamente mais leve e com menos parâmetros que as utilizadas na referência;
- **Resolução:** Redução da entrada para 224×224 px, diminuindo drasticamente a quantidade de informação visual processada;
- **Quantização:** Conversão dos pesos para inteiros de 8 bits (INT8), o que introduz ruído de quantização e limita a precisão numérica;
- **Pré-processamento:** Uso de técnicas de aumento de dados mais conservadoras, compatíveis com um fluxo de treinamento rápido e leve.

Dessa forma, a acurácia de aproximadamente 68% obtida no conjunto de validação não deve ser interpretada apenas como um déficit de desempenho, mas sim como a demonstração do compromisso entre precisão e eficiência. O resultado confirma que, mesmo com drásticas reduções de complexidade para adequação ao hardware embarcado, o sistema mantém capacidade de generalização funcional para a tarefa proposta.

6.1.3 Relatório de classificação por classe

O relatório por classe (precisão, sensibilidade e F1 por rótulo) é apresentado na Tabela 9. Os valores refletem o comportamento do classificador por categoria específica, auxiliando na identificação de classes com maior taxa de confusão.

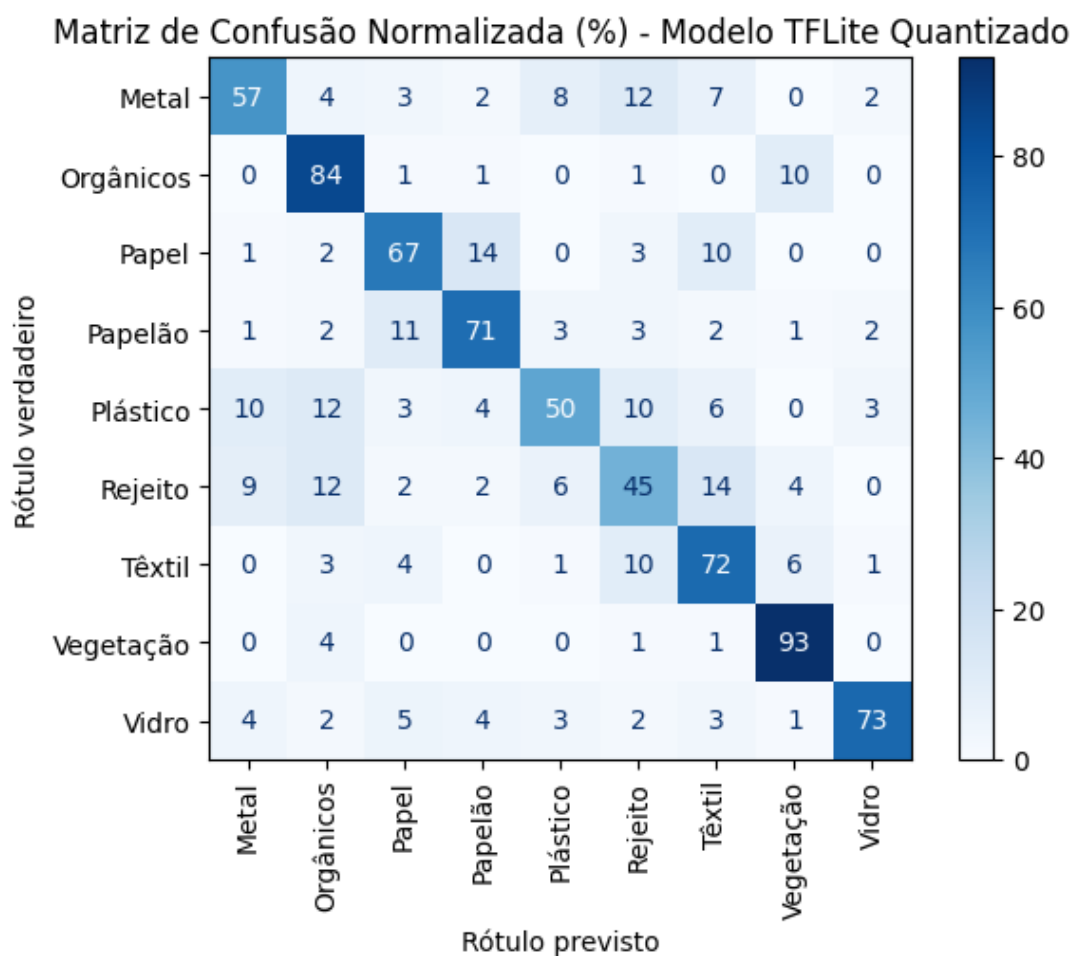
Tabela 9: Relatório de classificação por classe (validação).

Classe	Precisão	Sensibilidade	F1-score
Metal	0,67	0,58	0,62
Orgânicos	0,64	0,84	0,73
Papel	0,70	0,67	0,69
Papelão	0,70	0,72	0,71
Plástico	0,69	0,50	0,58
Rejeito	0,50	0,45	0,47
Têxtil	0,57	0,72	0,64
Vegetação	0,81	0,93	0,87
Vidro	0,88	0,74	0,80

Fonte: Do autor

6.1.4 Matriz de confusão

A Figura 7 apresenta a **matriz de confusão normalizada por classe** (em %), permitindo visualizar padrões de acerto/erro entre categorias. As diagonais indicam acertos; valores fora da diagonal revelam confusões recorrentes.

**Figura 7:** Matriz de confusão normalizada (%).

Fonte: Do autor

6.1.5 Curvas COR multiclasse

A Figura 8 mostra as **curvas COR por classe** e as respectivas áreas sob a curva (AUC), calculadas a partir dos escores de saída do modelo para cada rótulo. Em cenários multiclasse, cada curva contrasta o desempenho *um-versus-resto* de uma classe específica.

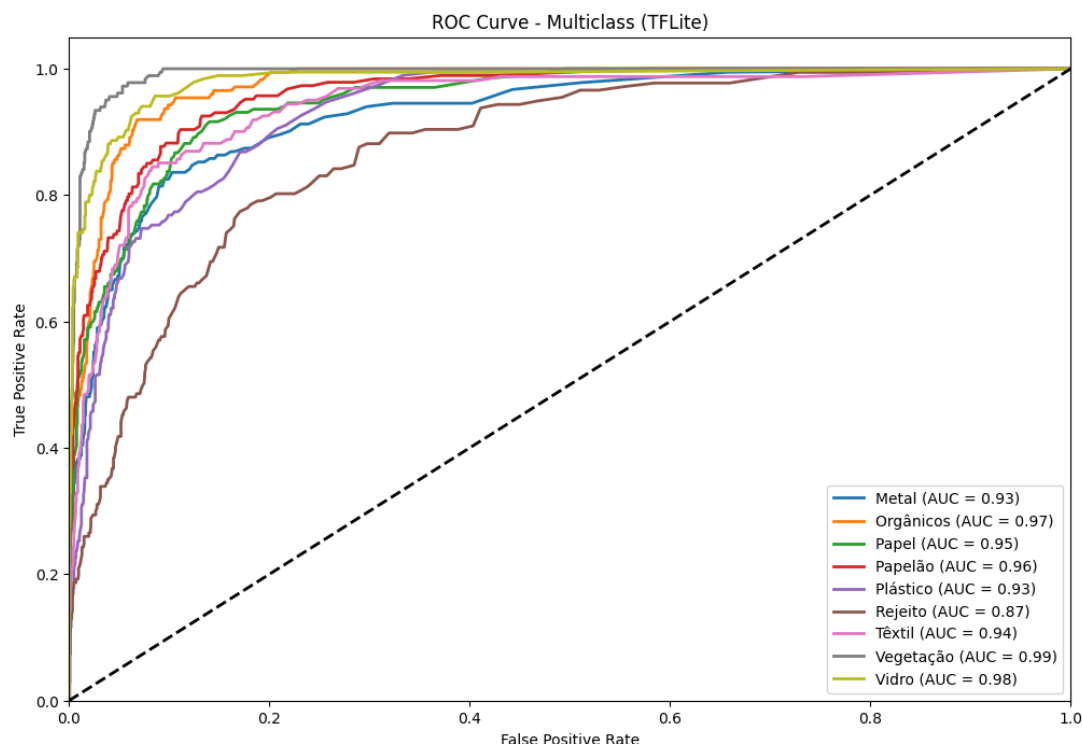


Figura 8: Curvas COR por classe e respectivas AUCs.

Fonte: Do autor

6.1.6 Discussão

Em linhas gerais, as médias ponderadas (PRE, SEN, F1) refletem o impacto do *suporte* de cada classe; assim, classes com menos amostras tendem a apresentar maior variância nas métricas individuais, influenciando menos o agregado ponderado. A leitura conjunta da matriz de confusão e das curvas COR permite identificar pares de classes mais propensos à confusão e orientar possíveis ajustes: coleta adicional para classes minoritárias, refinamento de aumento de dados direcionado e, se necessário, afrouxar ou ajustar limiares de decisão em cenários operacionais específicos.

6.2 AVALIAÇÃO E ANÁLISE DE EFICIÊNCIA COMPUTACIONAL

A eficiência computacional do sistema foi avaliada considerando três aspectos principais: (i) tempo médio de inferência por imagem, (ii) uso de CPU durante a execução do modelo e (iii) consumo de memória RAM. Todos os ensaios foram realizados utilizando a montagem descrita a seguir.

6.2.1 Montagem experimental

A Figura 9 apresenta a estrutura física utilizada para todos os ensaios. O suporte foi construído com tubos de PVC, permitindo fixar a webcam USB na posição superior, perpendicular ao plano de deposição dos resíduos. O Raspberry Pi utilizado para realizar as inferências foi instalado diretamente na estrutura, garantindo estabilidade e reduzindo interferências externas. Essa montagem assegurou distância fixa entre câmera e objeto, iluminação uniforme e reprodutibilidade dos testes. A fonte de iluminação para os ensaios é natural indireta.

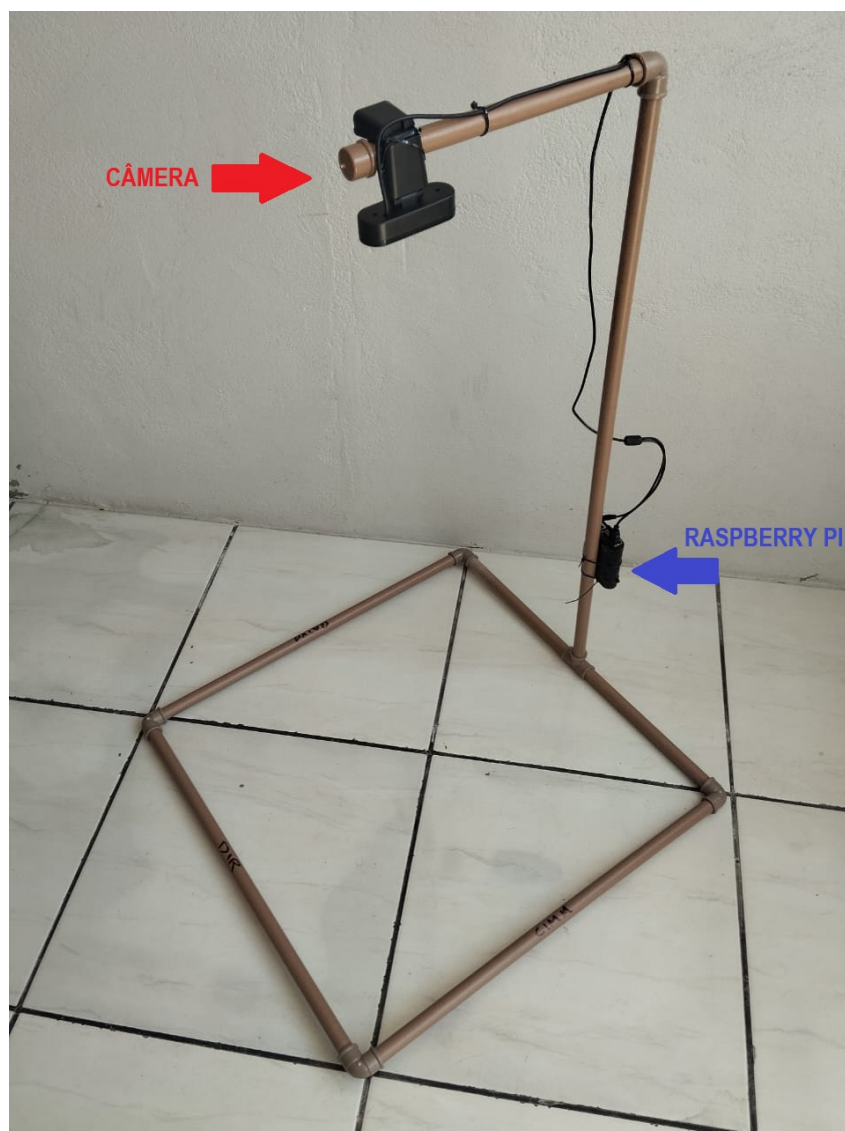


Figura 9: Montagem utilizada nos ensaios, composta por tubos de PVC, presilhas de fixação, webcam USB e Raspberry Pi.

Fonte: Do autor

A base da estrutura mede 60 cm × 60 cm, proporcionando área suficiente para a colocação dos resíduos a serem classificados. A altura da haste que suporta a câmera foi regulado experimentalmente, fixando o foco da câmera e ajustando a altura da câmera de forma a enquadrar adequadamente a área interna do quadrado da base. Com este teste

feito, a altura ideal encontrada para este caso foi de 90 cm. Finalmente, um tubo de 30 cm foi utilizado de forma a centralizar a câmera sobre a base.

A Tabela 10 detalha os materiais utilizados na construção da estrutura.

Tabela 10: *Lista de materiais para a estrutura de suporte (PVC 1/2").*

Item	Quantidade
Segmentos de 60 cm (base)	4
Segmento de 90 cm	1
Segmento de 30 cm	1
Joelhos (90°)	5
Conexão em T	1
Tampão (<i>cap</i>)	1

Fonte: Do autor

Nota: Para a montagem da base, o segmento de 60 cm deve ser seccionado para a inserção da conexão em T. Recomenda-se cortar o tubo inicialmente ao meio e acoplar as duas partes à conexão. Em seguida, deve-se realizar o ajuste final nas extremidades para que o conjunto (tubos + conexão) atinja o comprimento total exato de 60 cm.

6.2.2 Resultados sobre o tempo de inferência

As medições de latência e as métricas apresentadas nesta subseção foram obtidas a partir da rotina de teste desenvolvida no arquivo `metadados/test_tinymml_rpi.py`. O código carrega o modelo com `tflite_runtime.Interpreter`, captura quadros da câmera via OpenCV, redimensiona cada frame para o `input_shape` esperado pelo modelo e converte os dados para `uint8` quando se trata de modelo quantizado. A latência por inferência é medida com `time.perf_counter()` imediatamente antes e após `interpreter.invoke()` e convertida para milissegundos; além disso é computada uma média exponencial (EMA) da latência usando $\alpha = 0,12$ para suavizar variações pontuais. A aplicação registra ainda um log simples a cada 30 frames e sobrepõe no próprio frame a predição, a confiança (saída dividida por 255 para modelos `uint8`) e o tempo de inferência atual.

Durante os ensaios, observou-se que o sistema manteve tempos de inferência estáveis, próximos de 200 ms por imagem. A Figura 10 apresenta um exemplo do ensaio com um objeto metálico, onde o tempo de inferência registrado foi de aproximadamente 204 ms. Esse valor é representativo do comportamento geral do modelo durante todo o período de testes. Todos os ensaios foram realizados alocando os objetos de forma não ordenada, isto é de forma aleatória.

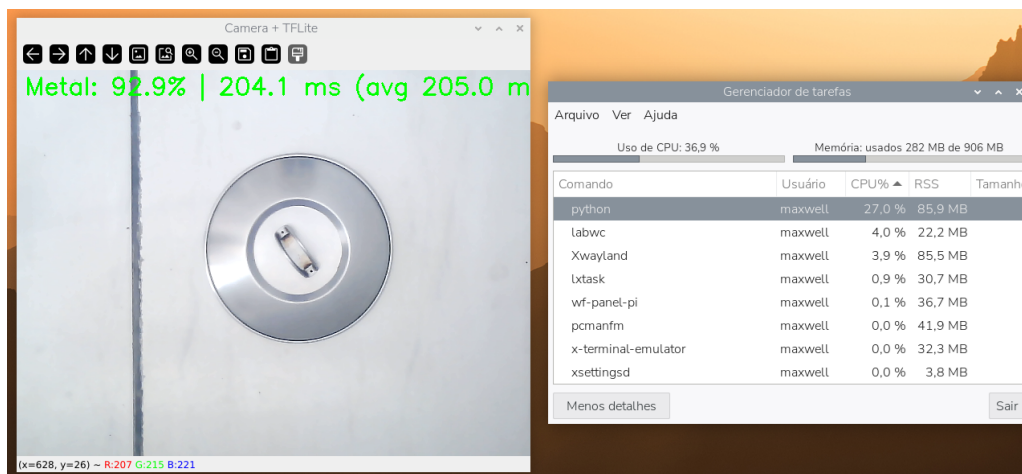


Figura 10: Ensaio com tampa metálico de $\varnothing$ 160 mm mostrando a predição da classe e o tempo de inferência.

Fonte: Do autor

Também foi avaliado o desempenho com resíduos da classe papel A4. A Figura 11 mostra que o sistema apresentou tempo de inferência da ordem de 220 ms, valor compatível com o observado nos demais ensaios.

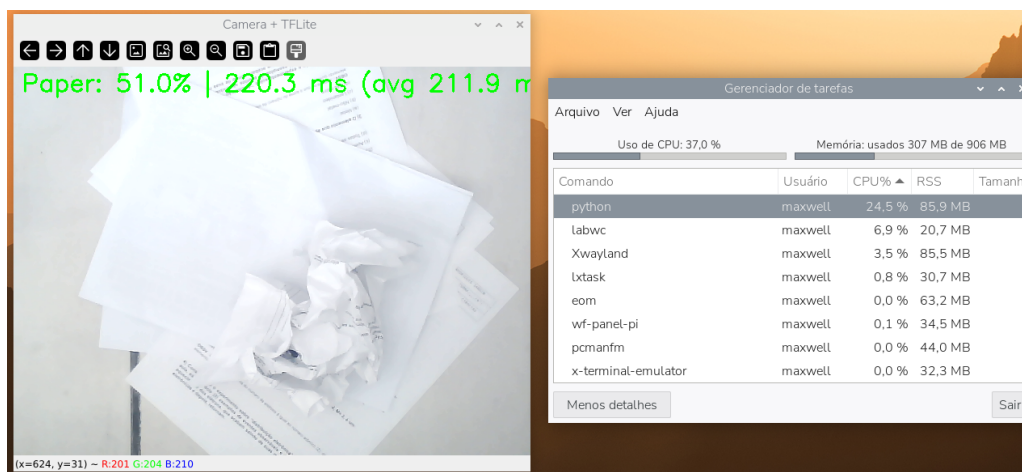


Figura 11: Ensaio com papel A4, ilustrando o tempo de inferência registrado e a classificação produzida pelo modelo.

Fonte: Do autor

A Figura 12 corresponde ao ensaio com papelão, com tempo de inferência em torno de 210 ms, mantendo o padrão observado nos demais testes e confirmando estabilidade temporal do modelo. As amostras de papelão são caixas de medicamentos.

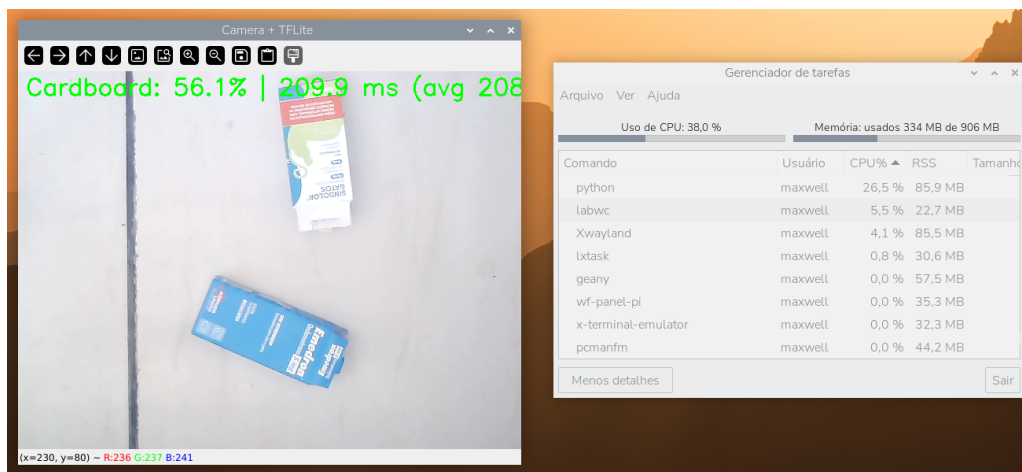


Figura 12: Ensaio com caixas de medicamento (papelão), exibindo o tempo de inferência e a predição realizada.

Fonte: Do autor

Por fim, a Figura 13 mostra o desempenho com material têxtil, novamente mantendo tempos de inferência próximos de 210 ms. Neste caso foi utilizada toalha de banho amassada.

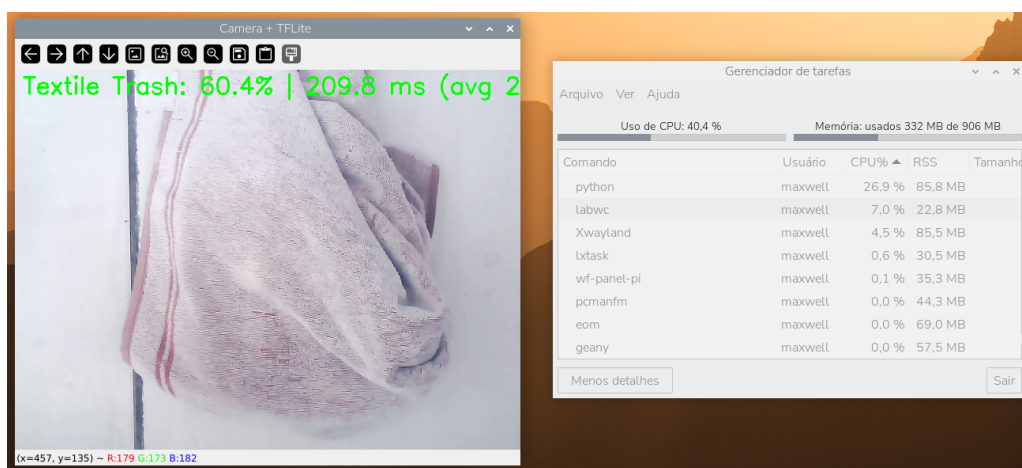


Figura 13: Ensaio com toalha de banho (têxtil), evidenciando tempo de inferência e classificação final.

Fonte: Do autor

6.2.3 Uso de CPU e memória RAM

Em todos os experimentos, o uso de CPU observado no processo `python` permaneceu entre 25% e 27%. Esse valor indica utilização praticamente completa de um dos quatro núcleos do Raspberry Pi, o que é esperado, pois o TensorFlow Lite, em sua configuração padrão, executa inferências utilizando apenas um núcleo de processamento.

O consumo de memória RAM apresentou valores entre 85 MB e 90 MB para o processo responsável pela inferência, enquanto o uso total do sistema permaneceu entre 280 MB e 330 MB. Esses valores confirmam que a aplicação opera de forma estável, sem risco de esgotamento dos recursos físicos do dispositivo.

6.2.4 Análise comparativa: Quantizado (INT8) vs. Ponto Flutuante (Float32)

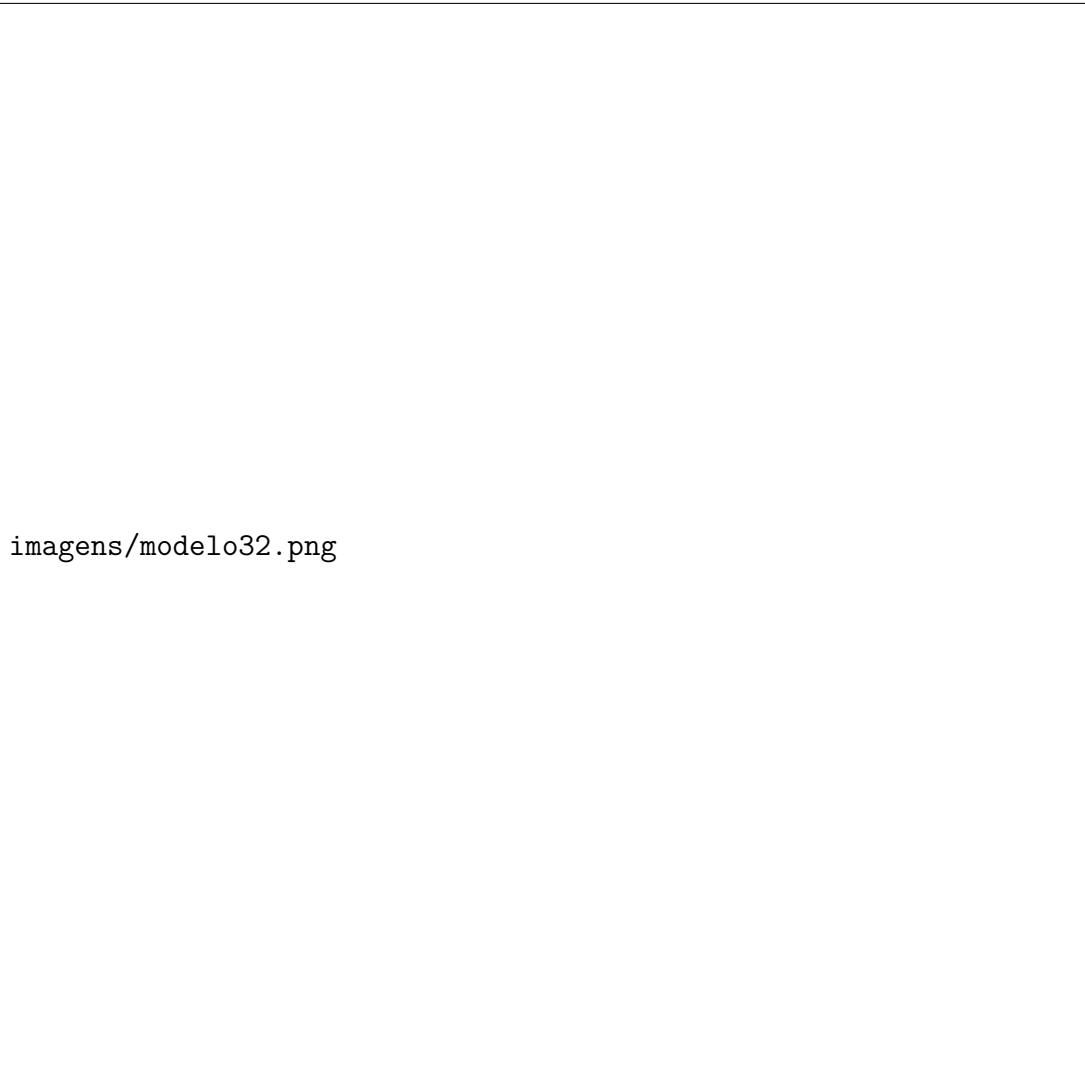
Com o objetivo de isolar e quantificar os ganhos reais trazidos pelo processo de quantização na plataforma Raspberry Pi 3 Model B+, realizou-se um experimento de controle executando o modelo original em ponto flutuante de 32 bits (*Float32*), mantendo inalteradas as demais variáveis do sistema (hardware, versão do interpretador e script de teste). Os resultados visuais e numéricos deste ensaio são apresentados na Figura 14.

A análise dos dados revela dois fenômenos distintos no comportamento do hardware:

1. **Latência de Inferência (Empate Técnico):** Contrariando a intuição inicial de que modelos inteiros seriam drasticamente mais rápidos, a latência média permaneceu virtualmente idêntica entre as versões, orbitando a faixa de 183 ms. Este comportamento é atribuído à arquitetura do processador Broadcom BCM2837B0 (Cortex-A53). Esta CPU possui unidades de ponto flutuante robustas e suporte a instruções vetoriais NEON (SIMD). O interpretador TensorFlow Lite, quando executado neste hardware via biblioteca XNNPACK, consegue otimizar operações de ponto flutuante com eficiência similar às operações inteiras, anulando o ganho de velocidade que seria evidente em microcontroladores sem FPU dedicada (como um Arduino ou ESP32).
2. **Consumo de Memória (Redução Seletiva):** A diferença crítica manifestou-se no consumo de memória RAM (RSS). O processo com o modelo *Float32* ocupou **107,9 MB**, enquanto a versão quantizada (apresentada na Seção 6.2) estabilizou em aproximadamente **85 MB**.

A diferença de aproximadamente **23 MB** valida a eficácia da quantização, mas também expõe o custo fixo da aplicação. Embora a quantização reduza o tamanho dos pesos e tensores do modelo em quatro vezes (de 32 para 8 bits), o consumo total não cai na mesma proporção. Isso ocorre porque o *footprint* basal do interpretador Python e das bibliotecas carregadas (TensorFlow Runtime, OpenCV, Numpy) representa um custo fixo imutável (o "chão" de consumo), sobre o qual o peso variável do modelo é adicionado.

Conclui-se, portanto, que para a arquitetura Raspberry Pi 3, a principal vantagem da quantização INT8 reside na eficiência espacial (economia de RAM e armazenamento) e não na aceleração temporal, liberando recursos valiosos do sistema para outros processos concorrentes.



imagens/modelo32.png

Figura 14: *Ensaio de controle com modelo Float32: latência equivalente (183 ms), porém com aumento significativo no consumo de memória (+27%) devido à precisão de 32 bits.*

Fonte: Do autor

6.2.5 Síntese dos resultados

Os resultados obtidos permitem concluir que o modelo MobileNetV2 quantizado executado via TensorFlow Lite apresenta desempenho adequado para operação embarcada em tempo real, com:

- latência média entre 205 ms e 215 ms por inferência;
- uso estável de um único núcleo de CPU (aprox. 27%);
- baixo consumo de memória RAM (cerca de 85 MB);
- estabilidade entre diferentes classes de resíduos.

Esses valores estão alinhados com trabalhos similares na literatura e confirmam que a solução do estudo de caso atende aos requisitos de desempenho definidos para o protótipo.

6.3 LIMITAÇÕES E SUGESTÕES DE MELHORIA

Durante o desenvolvimento e a análise dos resultados, algumas limitações do modelo puderam ser observadas. A matriz de confusão apresentada anteriormente já indicava a tendência do modelo em confundir determinadas classes, especialmente entre vidro, plástico e metal. Esse comportamento está alinhado com o valor do escore F1 obtido (aproximadamente 0,68), evidenciando que ainda há margem para melhorias na capacidade discriminativa do modelo.

Outro ponto notado foi a influência da iluminação e da forma como os objetos são posicionados durante a captura das imagens. Variações nesses fatores impactaram diretamente o desempenho, sugerindo que a padronização das condições de captura poderia contribuir para resultados mais consistentes.

Além disso, embora o uso de bancos de dados públicos tenha sido valioso, eles nem sempre se mostraram totalmente adequados para um problema tão específico quanto o abordado neste trabalho. A criação de um conjunto de imagens próprio, mais direcionado ao contexto real de uso, pode melhorar significativamente a qualidade do treinamento. Uma abordagem híbrida, combinando dados próprios com dados públicos selecionados, surge como alternativa mais robusta.

A qualidade e o tipo da câmera utilizada também se mostraram fatores relevantes. A câmera empregada apresentou limitações, como ruído visual e granulação, o que possivelmente afetou o desempenho do modelo. Para trabalhos futuros, recomenda-se utilizar a mesma câmera tanto para a construção do conjunto de treinamento quanto para a realização das inferências, ou empregar câmeras de maior qualidade.

Por fim, observa-se que a otimização dos parâmetros e hiperparâmetros do modelo pode ser uma linha promissora de evolução. Técnicas como ajuste sistemático de taxa de aprendizado, número de camadas treináveis, funções de ativação, além de estratégias de data augmentation específicas, podem contribuir para elevar significativamente o desempenho do modelo em aplicações reais.

7 CONCLUSÕES

O presente trabalho teve como objetivo geral avaliar a viabilidade da aplicação de técnicas de aprendizado de máquina em dispositivos embarcados com recursos computacionais limitados. Com base nos resultados obtidos e nas etapas desenvolvidas ao longo do estudo, é possível afirmar que esse objetivo foi plenamente atingido.

Os objetivos específicos definidos no Capítulo 1 também foram alcançados de forma consistente. Primeiramente, realizou-se uma pesquisa abrangente sobre ferramentas de desenvolvimento TinyML, destacando suas capacidades, limitações e adequação a diferentes cenários de uso. Em seguida, o levantamento e a análise das principais plataformas de hardware disponíveis permitiram contextualizar alternativas viáveis para implementação de soluções embarcadas, considerando fatores como desempenho, custo e consumo energético.

Na sequência, foi elaborado um fluxo de trabalho ponta-a-ponta para desenvolvimento de soluções TinyML, cobrindo desde a aquisição e preparação dos dados até a quantização, validação e implantação do modelo. Esse fluxo se mostrou adequado tanto conceitualmente quanto na prática, servindo como diretriz estruturada para projetos semelhantes.

Por fim, o quarto objetivo específico — o desenvolvimento de um modelo TinyML para classificação de resíduos sólidos — foi atingido de forma completa. O modelo implementado, baseado em uma MobileNetV2 quantizada em INT8, apresentou desempenho satisfatório dentro das restrições de hardware, com latência estável em torno de 200 ms e consumo reduzido de recursos computacionais. Além disso, foi possível compará-lo diretamente com o estudo de referência do conjunto de dados RealWaste, reforçando a coerência dos resultados e evidenciando os compromissos associados a aplicações embarcadas.

Os experimentos realizados demonstram que o uso de TinyML pode ser uma alternativa viável para sistemas que exigem processamento local, baixa latência e independência de conectividade, desde que os requisitos de latência da aplicação alvo sejam atendidos. Embora o desempenho obtido seja inferior ao de modelos executados em ambientes tradicionais de maior capacidade computacional, os resultados mostram que, para o contexto de recursos limitados da plataforma Raspberry Pi 3, soluções compactas e quantizadas podem operar de forma robusta e eficiente em dispositivos de baixo custo.

Como perspectivas de continuidade, sugere-se o aprimoramento do conjunto de dados, a adoção de câmeras de maior qualidade, a exploração de arquiteturas mais otimizadas para quantização e o ajuste sistemático de hiperparâmetros. Essas estratégias podem elevar a precisão do modelo e ampliar sua aplicabilidade em cenários reais.

Em síntese, o trabalho confirma que técnicas de TinyML podem substituir ou complementar arquiteturas tradicionais de IoT em aplicações práticas, especialmente em cenários onde a velocidade de inferência obtida é suficiente para o fluxo de trabalho, consolidando o potencial da computação de borda como elemento fundamental na evolução de sistemas inteligentes embarcados.

Assim, além de comprovar a viabilidade técnica para este estudo de caso, este estudo evidencia que aplicações TinyML representam um caminho concreto para a próxima geração de sistemas inteligentes embarcados que priorizam a eficiência energética e a autonomia de processamento, reforçando o papel de soluções compactas e eficientes na arquitetura tecnológica contemporânea.

REFERÊNCIAS

- AHMED, H. A. *et al.* An Investigation on Disparity Responds of Machine Learning Algorithms to Data Normalization Method. *ARO - The Scientific Journal of Koya University*, v. 10, n. 2, p. 29–37, set. 2022. DOI: 10.14500/aro.10970. Disponível em: <https://aro.koyauniversity.org/index.php/aro/article/view/970>.
- ALSHAMMARI, A. Implementation of Model Evaluation using Confusion Matrix in Python. *International Journal of Computer Applications*, v. 186, p. 42–48, nov. 2024. DOI: 10.5120/ijca2024924236.
- ALTHNIAN, A. *et al.* Impact of Dataset Size on Classification Performance: An Empirical Evaluation in the Medical Domain. *Applied Sciences*, v. 11, n. 2, 2021. ISSN 2076-3417. DOI: 10.3390/app11020796. Disponível em: <https://www.mdpi.com/2076-3417/11/2/796>.
- DATAVERSE, H. *Harvard Dataverse: An Online Repository for Research Data*. [S. l.: s. n.], 2025. Acessado em: 4 jun. 2025. Disponível em: <https://dataverse.harvard.edu>.
- DEMIR, E.; KORKMAZ, H. A Novel Monitoring Dashboard And Hardware Implementation Simplifying The Remote Access In Industry. *In: 2023 IEEE International Workshop on Metrology for Industry 4.0 & IoT (MetroInd4.0&IoT)*. [S. l.: s. n.], 2023. p. 399–403. DOI: 10.1109/MetroInd4.0IoT57462.2023.10180193.
- EDGE, G. A. *LiteRT: High-Performance On-Device AI Runtime (formerly TensorFlow Lite)*. [S. l.: s. n.], 2024. Acessado em: 4 jun. 2025. Disponível em: <https://ai.google.dev/edge/litert>.
- EDGE IMPULSE. *Image Classification - Edge Impulse Documentation*. [S. l.], 2025a. Acessado em: 7 jun. 2025. Disponível em: <https://docs.edgeimpulse.com/docs/tutorials/end-to-end-tutorials/computer-vision/image-classification>.
- EDGE IMPULSE, I. *Edge Impulse: The Leading Edge AI Development Platform*. [S. l.: s. n.], 2025b. Acessado em: 4 jun. 2025. Disponível em: <https://www.edgeimpulse.com>.
- ESSANOAH ARTHUR, E. A. *et al.* Edge Impulse vs TensorFlow: A Comparative Analysis of TinyML Platforms for Maize Leaf Disease Identification. *In: 2024 Conference on Information Communications Technology and Society (ICTAS)*. [S. l.: s. n.], 2024. p. 1–6. DOI: 10.1109/ICTAS59620.2024.10507119.

- FOUNDATION, R. P. *Raspberry Pi 3 Model B+*. Acessado em: 27 out. 2025. 2025. Disponível em: <https://www.raspberrypi.com/products/raspberry-pi-3-model-b-plus/>.
- GHOLAMI, A. *et al.* A Survey of Quantization Methods for Efficient Neural Network Inference. *arXiv preprint arXiv:2103.13630*, 2021. Disponível em: <https://arxiv.org/abs/2103.13630>.
- HAN, H.; SIEBERT, J. TinyML: A Systematic Review and Synthesis of Existing Research. *In: 2022 International Conference on Artificial Intelligence in Information and Communication (ICAIIIC)*. [S. l.: s. n.], 2022. p. 269–274. DOI: 10.1109/ICAIIIC54071.2022.9722636.
- JACOB, B. *et al.* Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference. *In: PROCEEDINGS of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. [S. l.: s. n.], 2018. p. 2704–2713. DOI: 10.1109/CVPR.2018.00286.
- KAGGLE. *Kaggle: Your Machine Learning and Data Science Community*. [S. l.: s. n.], 2023. Accessed: 2024-11-02. Disponível em: <https://www.kaggle.com>.
- KALLIMANI, R. *et al.* TinyML: Tools, applications, challenges, and future research directions. *Multimedia Tools and Applications*, v. 83, n. 10, p. 29015–29045, mar. 2024. ISSN 1573-7721. DOI: 10.1007/s11042-023-16740-9. Disponível em: <https://doi.org/10.1007/s11042-023-16740-9>.
- KJELLBY, R. A. *et al.* Long-range & Self-powered IoT Devices for Agriculture & Aquaponics Based on Multi-hop Topology. *In: 2019 IEEE 5th World Forum on Internet of Things (WF-IoT)*. [S. l.: s. n.], 2019. p. 545–549. DOI: 10.1109/WF-IoT.2019.8767196.
- MELLIT, A. *et al.* TinyML for Fault Diagnosis of Photovoltaic Modules Using Edge Impulse Platform and IR Thermography Images. *IEEE Transactions on Industry Applications*, p. 1–12, 2025. DOI: 10.1109/TIA.2025.3556792.
- MILLER, T. *et al.* AI in Context: Harnessing Domain Knowledge for Smarter Machine Learning. *Applied Sciences*, v. 14, n. 24, 2024. ISSN 2076-3417. DOI: 10.3390/app142411612. Disponível em: <https://www.mdpi.com/2076-3417/14/24/11612>.
- OLADOKUN, M. Latency and Throughput Tradeoffs in Real-Time IoT Data Analytics. *arXiv preprint arXiv:2505.XXXX*, maio 2025. Discusses desafios de latência em setores como saúde, transporte e manufatura em aplicações de IoT em tempo real.
- OLIVEIRA, F. *et al.* Internet of Intelligent Things: A convergence of embedded systems, edge computing and machine learning. *Internet of Things*, v. 26, p. 101153, 2024. ISSN 2542-6605. DOI: <https://doi.org/10.1016/j.iot.2024.101153>. Disponível em: <https://www.sciencedirect.com/science/article/pii/S2542660524000945>.

- PLÁSTICOS, E. *Serviços de seleção*. Acessado em: 07 dez. 2025. 2025. Disponível em: <https://www.ecopelplasticos.com.br/serv-selecao.php>.
- POOJARY, R.; PAI, A. Comparative Study of Model Optimization Techniques in Fine-Tuned CNN Models. *In: 2019 International Conference on Electrical and Computing Technologies and Applications (ICECTA)*. [S. l.: s. n.], 2019. p. 1–4. DOI: 10.1109/ICECTA48151.2019.8959681.
- RAINIO, O.; TEUHO, J.; KLÉN, R. Evaluation metrics and statistical tests for machine learning. *Scientific Reports*, v. 14, n. 1, p. 6086, 2024. ISSN 2045-2322. DOI: 10.1038/s41598-024-56706-x. Disponível em: <https://doi.org/10.1038/s41598-024-56706-x>.
- SARASWATHI, S. Optimizing Latency and Energy Efficiency in Edge Computing with Reinforcement Learning and TinyML. *In: 2025 International Conference on Emerging Smart Computing and Informatics (ESCI)*. [S. l.: s. n.], 2025. p. 1–8. DOI: 10.1109/ESCI63694.2025.10987923.
- SHWETHA, D.; SWETHA. IoT's Impact on Personal Devices and Health: Wearable Technology. *In: PROCEEDINGS of the 2024 International Conference on Advances in Modern Age Technologies for Health and Engineering Science (AMATHE)*. [S. l.: s. n.], 2024. p. 1–8. DOI: 10.1109/AMATHE61652.2024.10582194.
- SINGLE, S.; IRANMANESH, S.; RAAD, R. *RealWaste*. [S. l.: s. n.], 2023a. UCI Machine Learning Repository. DOI: <https://doi.org/10.24432/C5SS4G>.
- SINGLE, S.; IRANMANESH, S.; RAAD, R. RealWaste: A Novel Real-Life Data Set for Landfill Waste Classification Using Deep Learning. *Information*, v. 14, n. 12, 2023b. ISSN 2078-2489. DOI: 10.3390/info14120633. Disponível em: <https://www.mdpi.com/2078-2489/14/12/633>.
- SINGLE, S.; IRANMANESH, S.; RAAD, R. RealWaste: A Novel Real-Life Data Set for Landfill Waste Classification Using Deep Learning. *Information*, v. 14, n. 12, p. 633, 2023c. DOI: 10.3390/info14120633. Disponível em: <https://www.mdpi.com/2078-2489/14/12/633>.
- SINHA, S. State of IoT 2024: Number of connected IoT devices growing 13% to 18.8 billion globally, 2024. Disponível em: <https://iot-analytics.com/number-connected-iot-devices/>.
- TENSORFLOW. *Image Classification - TensorFlow Models*. [S. l.], 2023. Acessado em: 7 jun. 2025. Disponível em: https://www.tensorflow.org/tfmodels/vision/image_classification.
- WANG, Q. *et al.* A Comprehensive Survey of Loss Functions in Machine Learning. *Annals of Data Science*, v. 9, n. 2, p. 187–212, abr. 2022. ISSN 2198-5812. DOI: 10.1007/s40745-020-00253-5. Disponível em: <https://doi.org/10.1007/s40745-020-00253-5>.

- WHANG, S. E. *et al.* Data collection and quality challenges in deep learning: a data-centric AI perspective. *The VLDB Journal*, v. 32, n. 4, p. 791–813, jul. 2023. ISSN 0949-877X. DOI: 10.1007/s00778-022-00775-9. Disponível em: <https://doi.org/10.1007/s00778-022-00775-9>.
- WULNYE, F. A. *et al.* TinyML Implementation on Microcontrollers: The Case of Maize Leaf Disease Identification. *In: 2024 Conference on Information Communications Technology and Society (ICTAS)*. [S. l.: s. n.], 2024. p. 180–185. DOI: 10.1109/ICTAS59620.2024.10507115.
- XAVIER, T. C. S. *et al.* Managing Heterogeneous and Time-Sensitive IoT Applications through Collaborative and Energy-Aware Resource Allocation. *ACM Transactions on Internet of Things*, v. 3, n. 2, 10:1–10:28, maio 2022. DOI: 10.1145/3488248.
- YAP, Y. S.; AHMAD, M. R. Modified Overcomplete Autoencoder for Anomaly Detection Based on TinyML. *IEEE Sensors Letters*, v. 8, n. 10, p. 1–4, 2024. DOI: 10.1109/LSENS.2024.3463977.
- ZEN, K. *et al.* Latency Analysis of Cloud Infrastructure for Time-Critical IoT Use Cases. *In: 2022 Applied Informatics International Conference (AiIC)*. [S. l.: s. n.], 2022. p. 111–116. DOI: 10.1109/AiIC54368.2022.9914601.